ChatGPT

Редизайн dtf.twocomms.shop — техническое задание

1. Общая концепция сайта

Визуальная философия: Дизайн объединяет концепции **«Control Deck»** и **«Lab Proof»** – интерфейс одновременно выглядит как высокотехнологичная панель управления и как экспериментальная лаборатория. Это означает строгость структуры и **прозрачность процесса**, визуально показывающая надежность технологии. Главный акцент – **фирменная фишка Printhead Scan** (имитация прохода печатающей головки по изображению). Вокруг нее строится вау-эффект, но общий стиль остается сдержанным, без лишнего визуального шума. Каждый элемент на экране оправдан и работает на цель, ничего случайного.

WOW-дизайн vs конверсия: Мы стремимся произвести «вау»-впечатление, **не жертвуя понятностью и эффективностью**. Яркие визуалы должны привлекать внимание, а четкая структура – удерживать и вести к целевому действию. Как отмечают специалисты, **одна красота без ясного месседжа не конвертирует** – эффектный сайт проваливается, если страдает понятность или анимация мешает восприятию [1]. С другой стороны, сугубо утилитарный дизайн без эмоции не запомнится пользователю. Баланс достигается сочетанием эмоционального входа и логичного, простого продолжения: **«эмоция без ясности — это шум, а ясность без эмоции — невидима»** [2]. Поэтому мы даем пользователю **яркий первый экран** с одной впечатляющей деталью и понятным призывом, а дальше – только необходимое по сути. **Минимализм усиливает эффект, когда каждый элемент оправдан: сдержанность часто воспринимается более премиально, чем перегруженность** [3]. Пустое пространство рассматриваем как часть дизайна – своего рода «ритм и доверие» для глаза.

Основной герой (Hero): Главная страница с первых секунд должна вызывать «вау». Здесь используется единый сильный визуальный образ + **лаконичный слоган** + **чёткий CTA**. Например, на фоне может идти тонкая технологичная анимация (контролируемая **Printhead Scan** или абстрактный паттерн из мира печати), сразу демонстрируя дух инновации. Но сообщений минимум: одна ключевая фраза о продукте и кнопка действия. Пользователь не должен растеряться – только заинтересоваться. Далее прокрутка ведет его по секциям с логичным повествованием о сервисе.

Без визуального шума: Подход «Lab Proof» подразумевает доказательство качества и технологичности без излишков. В дизайне **запрещены отвлекающие элементы**, не несущие функции. Нет бессмысленных декораций, анимаций ради анимации, перегруженных фонов. Каждый эффект должен либо усиливать понимание, либо создавать эмоцию, но **не мешать восприятию контента**. Если у нас есть динамические «вау»-элементы, они должны направлять внимание, а не рассеивать его. Иначе креативность превратится в хаос, пользователь потеряет доверие и не совершит целевое действие [1]. Таким образом, **правило одной signature-фичи**: главная фишка – Printhead Scan – привлекает и запоминается, а всё остальное поддерживает её, сохраняя удобство и конверсию.

Производительность и UX: Быстродействие и отзывчивость интерфейса – неотъемлемая часть впечатления. Сайт должен ощущаться **«быстрым и живым»**, чтобы усилить чувство уверенности у пользователя. Скорость загрузки и плавность работы – это тоже эмоция: **быстрый сайт внушает доверие, медленный сеет сомнение** [4]. Поэтому все графические элементы оптимизированы, анимации – максимально легковесные. Мы используем современные технологии (напр. CSS-трансitions, Web Animations API) так, чтобы **интерфейс двигался грациозно, но не был «тяжелым» для устройства** [4]. Контроль перформанса – часть концепции: впечатлять, **не вызывая фрустрации из-за лагов**. Любой сложный эффект сопровождается планом деградации для слабых устройств (подробно об этом – в секции ограничений).

<!-- Codex: Ensure the overall design philosophy is clear – one standout WOW element (Printhead Scan), otherwise minimal, focused design that balances emotion and clarity. -->

2. Архитектура страниц

Каждая ключевая страница имеет свое назначение и правила оформления. Ниже описаны основные страницы, их цели, визуальный ритм и допустимые/недопустимые эффекты.

Главная страница (Home)

- **Цель:** Презентация сервиса и мгновенное вовлечение. Донести уникальность предложения (DTF-печать) и направить к началу заказа.
- **Визуальный ритм:** Самый динамичный раздел сайта. Hero-блок – **максимальный акцент** (вплоть до Motion tier L3–L4, см. ниже) с фирменным вау-эффектом. Далее секции – более спокойные (L1–L2): описание услуги, преимущества, возможно, отзывы. Ритм чередует яркие блоки с «воздухом» (пустым пространством) для передышки глаз. Информация подается порционно, экран за экраном, чтобы не перегружать.
- **Разрешённые эффекты:** В Hero-блоке – **Printhead Scan** или другой единичный WOW-элемент уровня L4 для привлечения внимания. Допустимы плавные появления блоков при прокрутке (fade-in, slide-in на L2) для поддержания ощущения современности. Легкие параллакс-скроллы или изменение фона (L2) могут использоваться, но умеренно. Наводя курсор на CTA или ключевые элементы – **микро-анимации L1** (подсветка кнопки, небольшое смещение иконки). Все эффекты должны направлять взгляд: например, стрелка, слегка покачивающаяся, указывая на кнопку «Заказать».
- **Запрещённые эффекты:** На главной нельзя иметь **несколько соревнующихся между собой анимаций**. Только одна доминанта. Запрещены слишком навязчивые эффекты (например, вспыхивающие баннеры, автопроигрывающиеся видео со звуком). Нельзя перегружать пользователя: если есть видео-фон, поверх него не должно быть анимированного текста и трех разных движущихся объектов одновременно. Также избегаем длительных интро-анимаций, задерживающих появление полезного контента – вау-эффект не должен ухудшать показатель LCP и первый экран. На слабых устройствах или при плохом соединении Hero-эффект должен **упрощаться или пропускаться** (фоллбек: статичный ключевой кадр вместо сложной анимации).

Страница заказа (Order)

- **Цель:** Конверсионная страница для оформления заказа DTF-печати. Пользователь выбирает параметры, загружает макет, просматривает превью (Preflight) и подтверждает заказ. Основная задача – **простота и понятность процесса**, минимальное трение до нажатия «Отправить заказ».

- **Визуальный ритм: Спокойный, сосредоточенный на форме.** Здесь приоритет UX: интерфейс похож на **контрольную панель (Control Deck)** – все нужные поля и шаги на виду, ничего лишнего. Ритм статичный (tier L0–L1): страницы заказа не должны отвлекать. Разделение на понятные шаги или секции (напр. “Загрузка файла”, “Параметры печати”, “Доставка/оплата”). Возможен пошаговый процесс (wizard) с минимумом визуальных эффектов, чтобы пользователь не потерялся.
- **Разрешённые эффекты: Микро-эффекты уровня L1** для интерактивности и обратной связи. Примеры: подсветка рамки поля ввода при фокусе, небольшое всплытие placeholder или метки поля (label) при вводе (материал-дизайн эффект). Кнопки могут иметь легкую анимацию нажатия (например, уменьшение размера на 0.95 при клике – ощущение нажатия). Допустим **прогресс-бар или таймлайн** шагов оформления заказа, который при переходе на следующий шаг плавно заполняется (анимация прогресса L2, быстрая и ненавязчивая). Если используется превью загруженного изображения, оно может появляться с коротким fade-in. Все динамики – исключительно для подтверждения действий пользователя и улучшения UX (например, спиннер загрузки при аплоаде файла).
- **Запрещённые эффекты:** Любые отвлекающие анимации, которые не связаны с непосредственным взаимодействием. Никаких постоянно мигающих элементов, фоновых видео или сложных трансформаций. **Printhead Scan не применяется** на этой странице (если только в небольшом превью в рамках проверки, но не на весь экран). Также запрещены эффекты, замедляющие ввод данных: например, анимированные переходы между полями формы (поля должны появляться мгновенно, без задержек). Минимизируем использование blur/тени на форме, чтобы не нагружать GPU на слабых устройствах. В целом, дизайн страницы заказа – **максимально предсказуемый и стабильный** визуально.

Страница статуса заказа (Status)

- **Цель:** Информировать пользователя о текущем статусе его заказа (принят, в печати, отправлен, выполнен и т.д.). Повысить уверенность пользователя, давая прозрачность процесса исполнения. Также здесь могут быть детали заказа, трекинг доставки.
- **Визуальный ритм: Статичный (L0–L1)**, информативный. Страница статуса напоминает дашборд или отчет: основное внимание – на **таймлайне статусов** или списке этапов. Возможен вертикальный или горизонтальный таймлайн с чекпоинтами, который пользователь просматривает сверху вниз. Визуальный ритм размеренный: элементы располагаются с равным интервалом (например, шаги пронумерованы или с датами). Никаких резких изменений – статус меняется асинхронно (например, при обновлении страницы или через авто-обновление).
- **Разрешённые эффекты: Легкие анимации подтверждения (L1–L2)** при обновлении статуса. Например, когда заказ переходит в следующий этап, соответствующий элемент таймлайна может подсветиться или появиться галочка с коротким появлением. Можно использовать цветовую индикацию: текущий этап подсвечен основным цветом, завершенные – помечены галочкой с затуханием. Допустима **анимация заполнения шкалы** (progress) в пределах таймлайна, если это уместно, но очень короткая. Если страница сама опрашивает сервер на наличие обновлений (через HTMX или WebSocket), при поступлении нового статуса можно плавно обновить текст/иконку (например, плавное изменение цвета или слайд-вверх старого статуса, слайд-вниз нового). Все эффекты направлены на **удобочитаемость**: привлекаем внимание к новому статусу, но не более.
- **Запрещённые эффекты:** Никаких отвлекающих баннеров или рекламных блоков на странице статуса – пользователь уже совершил целевое действие, ему важно спокойствие и уверенность. Не используем сложные анимации фона. Таймлайн не должен прыгать или прокручиваться сам по себе. Также нежелательны длительные CSS-анимации, которые

могут мешать быстро посмотреть статус (например, анимированная стрелка, которая едет 5 секунд – это лишнее). **Важно:** если пользователь открывает страницу на мобильном или слабом интернете, она должна сразу показать текст статуса без задержек – поэтому никаких больших видео или heavy-script на этом экране.

Галерея работ (Gallery)

- **Цель:** Продемонстрировать примеры готовых работ, вдохновить пользователя и подтвердить качество печати. Это витрина реальных кейсов: фотографии изделий с принтами, макеты и их реализованное воплощение. Также галерея усиливает доверие (показывая, что компания уже много сделала).
- **Визуальный ритм: Динамика умеренная (L1–L2)**, ориентированная на **визуальный контент**. Галерея представляет собой сетку или ленту изображений. Пролистывание/прокрутка – основное действие, поэтому ритм должен быть плавным, **скроль должен чувствоваться легким**. Раскладка может быть плиткой (grid) или горизонтальным слайдером. Между группами работ можно вставлять **пробелы (whitespace)** для разделения серий, чтобы глаза не уставали от массы картинок. Если есть категории или фильтры, они должны мгновенно перерисовывать галерею без резких вспышек (используем плавные фильтрации).
- **Разрешённые эффекты: Lazy-load с плавным появлением** изображений (fade-in на L1) по мере прокрутки, чтобы улучшить загрузку и добавить мягкий эффект. Ховеры на карточках работ: легкое увеличение изображения или проявление деталей (например, название работы накладывается при наведении с прозрачностью). Можно применять **несильный параллакс**: при скролле страницы изображения могут еле заметно смещаться относительно фона (L2), создавая глубину – но только если это не мешает пролистыванию. Допустим режим **сравнения/линзы** для отдельных кейсов: например, при клике на работу появляется модальное окно с инструментом сравнения (оригинальный макет vs фото печати) – об этом ниже. В целом эффекты в галерее должны служить тому, чтобы пользователь дольше разглядывал работы: небольшие **механики вовлечения**, вроде кнопки «Еще» которая при прокрутке автоматически подгружает новые изображения (можно с аккуратной индикатор-анимацией).
- **Запрещённые эффекты:** Галерея не должна самопроизвольно крутиться или играть слайдшоу без участия пользователя – **только пользователь контролирует** просмотр. Избегаем тяжёлых фильтров на много изображений (например, постоянный CSS-фильтр blur на 50 миниатюрах – это удар по производительности). Также нежелательны сложные тени или бликующие эффекты на preview картинках – они должны быть максимально чёткими. Не используем одновременно более 1–2 анимаций: например, если уже есть hover-скейл, то дополнительно крутить иконку «увеличить» рядом не стоит. И никаких всплывающих подсказок на каждое изображение, что может замусоривать интерфейс. Ключевое – **фокус на самих работах**, UI должен быть нейтральным.

Preflight (Предпечатная проверка)

- **Цель:** Специальная страница/раздел для **проверки загруженного макета перед печатью**. Здесь пользователь видит превью своего изображения на типовом фоне (например, на прозрачной пленке или на выбранном цвете изделия), получает предупреждения о потенциальных проблемах (разрешение, цвета, размеры) и подтверждает, что всё в порядке. Это последний шаг перед добавлением заказа в корзину или оплатой – важно уверить, что результат будет корректным.
- **Визуальный ритм: Сосредоточенный, но допускающий один вау-элемент (L2–L3)**, поскольку это тоже часть «лаб-проф» опыта – показ технологичности. Основной экран Preflight – крупное превью макета. Вокруг – детали: параметры печати, настройки

(например, выбор материалов, размеры), кнопки «Назад для правки» и «Ок, в заказ». Ритм визуальный такой: превью = центр внимания, под ним/справа от него – статичные поля. **Никакой прокрутки**, всё в одном экране (если не влезает – внутри блока превью может быть скролл).

- **Разрешённые эффекты: Эффект "Printhead Scan" на превью – фирменная фишка** здесь служит делу: по нажатию кнопки или автоматически, макет может быть **"просканирован" движущейся полосой**, которая имитирует печать. Например, белая или светящаяся линия проходит сверху вниз по изображению, постепенно проявляя финальное изображение (эффект «печать на глазах»). Это вау-эффект уровня L3, который одновременно выполняет функцию проверки: пользователь видит, как лягут слои краски. *Важно:* этот эффект должен выполняться быстро (пару секунд) и иметь возможность пропуска. Помимо него, **допустим инструмент сравнения**: пользователь может переключать вид превью – например, "с подложкой / без подложки" (если печать на темной поверхности, см. "симуляция подложки" ниже). Тут же уместна **"линза" сравнения**: часть изображения под лупой показывает, как будет выглядеть результат на материале (с учетом текстуры или подложки). Эта линза может перемещаться по изображению (L2 эффект, управляемый пользователем). Все предупреждения (например «низкое разрешение») могут появляться плавно с миганием иконки предупреждения (короткая анимация L2, привлекающая внимание к проблеме). Кнопка подтверждения при навидении может слегка пульсировать (едва заметно, L1) после завершения всех проверок, направляя пользователя нажать.
- **Запрещённые эффекты:** Нельзя автоматически запускать тяжелую анимацию, **если файл очень большой** – должна быть проверка перформанса. Если Printhead Scan может тормозить (большой макет на старом устройстве), лучше показать статичное превью и предложить «Просмотреть имитацию печати» кнопкой. Также запрещены любые эффекты, искажающие реальное восприятие макета: напр., чрезмерный **blur или фильтры** на превью (пользователю нужен честный вид). Не должно быть навязчивых таймеров или всплывающих окон поверх превью. И никакой рекламы здесь – пользователь должен полностью сфокусироваться на своем макете. Если устройство слабое или браузер старый, сложные эффекты (сканирование, 3D-превью) **должны отключаться, предлагая простой статический просмотр**.

Прочие страницы

Кроме основных, на сайте могут быть вспомогательные страницы (профиль пользователя, информация о сервисе, контакты, ошибки). Их дизайн следует общей системе: - **Профиль/Аккаунт:** Чистый UI, близкий к странице заказа (форма, список заказов). Минимум анимаций, разве что небольшие индикаторы.
- **Информационные страницы (о бренде, контакты):** Стиль как на главной (но без крупных вау-эффектов). Можно использовать общие компоненты и легкие fade-in при прокрутке.
- **Страница ошибки (404/500):** В духе проекта, может содержать креативное изображение или шутку, но не перегружена. Можно небольшую анимацию соответствующую (например, сломанный робот с мигающим индикатором – L2, очень легкая). Главное – дать понятные ссылки для выхода (на главную, и т.п.).
- **Мобильная версия:** Все страницы адаптируются под мобильные устройства. Эффекты на мобильной могут быть урезаны на один уровень вниз (например, вместо L3 – L2), чтобы не страдала производительность. Вертикальный скролл должен оставаться плавным, без "дерганья" из-за тяжелых скриптов.

<!-- Codex: Each page's guidelines above indicate where to apply heavy effects and where to keep it simple. Use these when structuring page templates and applying animations. -->

3. Motion tiers (уровни анимации L0–L4)

Все анимации и эффекты делятся на **5 уровней интенсивности**. Это системный подход для контролируемого применения WOW-эффектов без ущерба UX. Ниже описаны tier'ы и рекомендации их использования:

- **L0 – Статично:** Полное отсутствие движения. Используется для критически важных элементов и контента, требующего максимальной читабельности (например, текст условий, важные уведомления, кнопка оплаты во время ввода данных). На уровне L0 мы показываем абсолютную стабильность – ничего не мерцает и не двигается. *Применение:* базовое состояние большинства текстовых блоков, форм; режим **prefers-reduced-motion** (если пользователь отключил анимацию, весь сайт работает в L0).
- **L1 – Микро-анимации:** Едва заметные, контекстные изменения при взаимодействии. Не привлекают внимание сами по себе, а лишь дают тактильный отклик. *Примеры:* подсветка кнопки или ссылки при hover, изменение оттенка при фокусе, лёгкая вибрация иконки на кнопке при наведении, тень у карточки слегка усиливается при наведении. Длительность очень короткая (100–200мс), без задержек. *Применение:* состояние наведения/нажатия интерактивных элементов, мелкие индикаторы (напр., копируется текст – иконка чуть подпрыгнула). **L1 разрешён везде**, даже на самых строгих страницах, так как не мешает чтению и улучшает UX.
- **L2 – Малые переходы:** Небольшие анимации появления/исчезновения и перетекания состояний, заметные, но всё ещё быстрые. *Примеры:* плавное возникновение модального окна (fade/scale), скролл-индикатор, анимация прогресса загрузки, смена цвета кнопки при успехе (зелёная вспышка «готово»), слайд-шоу или переключение вкладок с исчезновением-появлением контента. Длительность умеренная (200–400мс), возможна лёгкая задержка для драматургии (до 100мс). *Применение:* точки, где нужно привлечь внимание к изменению состояния (например, добавление товара в корзину – значок корзины чуть дернулся), появления элементов при скролле (в пределах разумного). **L2 можно использовать широко**, кроме случаев, где пользователь может торопиться (например, форма оплаты – там минимально).
- **L3 – Средний WOW:** Более заметные эффекты, которые пользователь явно видит как анимацию, но они оправданы содержанием. *Примеры:* переход между страницами с помощью View Transition API (плавное перетекание героев страниц), сложное появление блока (например, карта, которая разворачивается), **Printhead Scan в действии** (если он показан быстро), параллакс с большей амплитудой, анимации на фоне (например, сетка точек медленно движется). Длительность может быть больше (0.5–1с), могут присутствовать более сложные кривые (easing типа ease-in-out для плавного начала/конца). *Применение:* ключевые моменты пользовательского пути, где уместно «удивить», но не задержать. Например, загрузка галереи – можно сделать L3 анимацию раскладки изображений. **L3 следует использовать точечно**, не более 1-2 элементов одновременно на экране. На страницах типа Home можно несколько L3 в разных секциях (но не в одном просмотре одновременно). В формах и процессах L3 обычно избыточен.
- **L4 – Большой WOW:** Максимально сложные и эффектные анимации, которые используются очень выборочно. Это то, что действительно выделяет бренд, но потенциально сильно нагружает внимание (и ресурсы системы). *Примеры:* полный **симулятор печати** на экране (Printhead Scan, рисующий изображение построчно), 3D-анимации, сложные последовательные эффекты из нескольких шагов (например, при переходе на сайт – камера «пролетывает» сквозь интерфейс). Часто L4 комбинирует несколько объектов и длится дольше секунды, возможно содержит звук (в нашем случае звук вряд ли, но на уровне идеи). *Применение:* **только один элемент на ключевом экране**. У нас это – фирменный Printhead Scan на главной или в Preflight. Больше L4

одновременно нигде не должно быть. Даже на главной: либо printhead-скан, либо, скажем, видеофон – но не вместе. **L4 по умолчанию отключён** на мобильных устройствах и слабых ПК (должен быть фоллбек на более простой эффект). Этот уровень – для десктопа с хорошей производительностью, где он действительно произведет впечатление.

Правила использования tier'ов: Во всех макетах помечаем, какой максимальный уровень анимации допустим для каждого блока. **В критически важных для конверсии местах (форма заказа, оплата)** устанавливаем потолок L1 (не выше). **В информативных, но не ключевых – L2. На вовлекающих промо-участках – L3,** и **L4 резервируем для точечных WOW-моментов** (главный экран, специальные демонстрации). Также учитываем настройки пользователя: если у него включена опция «Reduce Motion» (предпочтение уменьшить анимации), то **не показываем L3–L4 вовсе**, а L1–L2 сводим к мгновенным изменениям (можно в CSS @media (prefers-reduced-motion) отключить трансформы и transition-delay). Наш движок должен проверять это и переключать дизайн в более статичный режим при необходимости.

Кроме того, следует тестировать комбинации: например, если на странице несколько L2/L3 анимаций происходят при скролле, они не должны начинаться синхронно и вызывать фрезы. **Анимационный бюджет**: не более **2 одновременно движущихся объектов** на экране в любой момент (исключая совсем мелкие L1 реакции). И каждую анимацию останавливать/удалять, когда она выполнила задачу (не оставлять скрытых CSS animation, нагружающих CPU). Если страница простаивает, всё должно быть статично (никаких бесконечных анимированных фонов, кроме может быть очень легкого, еле заметного шумового движения на минимальном fps, если это концептуально).

<!-- Codex: Implement motion based on these tiers. Use CSS classes or data attributes to tag components with allowed motion level, and ensure to respect prefers-reduced-motion settings. -->

4. Компоненты UI

Сайт строится на основе унифицированных UI-компонентов. Каждый компонент должен соответствовать дизайн-системе, иметь состояния для взаимодействия, обеспечивать доступность и быть оптимизированным. Ниже перечислены основные компоненты и требования к ним.

Кнопки (Buttons)

- **Стиль:** Используются **прямоугольные кнопки с скруглением углов** средней степени (радиус ~4–6px, чтобы слегка смягчить форму, но сохранить техничность). Цвет кнопки – **основной акцентный** из палитры (см. визуальные материалы), текст контрастный (чаще всего белый на цветной кнопке или чёрный на светлой). Толщина границ: без рамки или тонкая линия, в зависимости от стиля (Control Deck склоняется к **ghost-buttons** с обводкой, Lab Proof допускает заполненные кнопки – итоговое решение: для первичных действий используем заполненную кнопку, для вторичных – обводку). Размер – удобный для нажатия: минимальная высота 40px (мобилы – 48px). Иконки на кнопках возможны (например, стрелка), но минимально и только если ясно дополнит надпись.
- **Состояния:** Каждая кнопка имеет состояния: *обычное*, *hover/фокус*, *нажатое (active)*, *неактивное (disabled)*. В обычном состоянии – стандартный вид. **Hover/Focus:** лёгкое повышение яркости или инверсия текста/фона на 5–10% для визуального отклика; курсор pointer; фокус (через клавиатуру) – помимо цвета, отображается **четкая обводка (outline)** контрастного цвета (например, тень или обводка 2px по периметру, цветом бренда или чёрно-белым, чтобы видно на любом фоне) для доступности. **Active (нажатие):** имитация

нажатия – кнопка словно уходит вглубь: уменьшается на пару пикселей (или scale ~0.98), тень пропадает или становится внутрь, цвет может стать чуть темнее. После отпускания возвращается в hover. **Disabled:** пониженная контрастность (например, светло-серый фон, серый текст) и курсор default/неактивный; никаких hover-эффектов.

- **Анимации:** На кнопках – **только L1** микро-анимации. Hover и active состояния меняются с небольшой анимацией (transition 0.1–0.2s) по свойствам background-color, color, box-shadow, transform. Возможно добавить еле заметный эффект при наведении: например, легкий **подъем кнопки** (тень чуть усилилась, кнопка "парит") – реализуется через box-shadow и y-translate(-2px). При нажатии – быстрый "щелчок" (scale down, затем up). **Ripple-эффект** (волна) при клике не используется, т.к. он может быть лишним шумом, вместо этого отдаём предпочтение простому scale, что даст тактильность. Для важных первичных кнопок можно добавить **световой индикатор**: например, если кнопка ведет к чему-то критичному, можно при hover подсвечивать её границу glow-эффектом (но очень слабым).
- **Доступность:** Все кнопки – настоящие `<button>` или `<a role="button">` с возможностью активации клавишей Enter/Space. Фокус-стиль не отключаем (или заменяем своим видимым). Цвета проверяем на контраст (минимум 4.5:1 между текстом и фоном кнопки). Текст на кнопке должен явно описывать действие (никаких одиночных иконок без `aria-label`). Если кнопка открывает модал или выполняет необратимое действие, можно предусмотреть `aria-describedby` для пояснения (например, «это удалит файл»). Для иконок в кнопке добавляем `aria-hidden="true"` и sr-only текст, если иконка несет смысл.
- **Производительность:** Кнопки – часто используемый элемент, должен быть максимально простой в DOM/CSS. Избегаем тяжелых теней (если используем тень при hover, она небольшая и размытие не огромное). Не используем на кнопках дорогие фильтры (blur, backdrop-filter) постоянно. Анимации делаем через transform/opacity для GPU оптимизации. Если на странице много кнопок (например, список товаров), **убедиться, что hover-эффекты не просаживают FPS**: можно использовать will-change на таких элементах (но осторожно, только во время hover, чтобы не удерживать лишний слой). В целом, стили кнопок должны браться из CSS переменных (темизация) и легко перекрашиваться без перерисовки изображений.

Поля ввода (Input fields)

- **Стиль:** Поля форм – **минималистичные, с четкими границами**. Фон – обычно светлый (белый или светло-серый), чтобы текст внутри был читаем. Рамка 1px сероватая (#ccc) или использование подчеркивания снизу вместо рамки (более современный подход, как в Material Design). Скругление углов небольшое (как и у кнопок, 4px, или вообще прямые углы для строгости – решается единообразно для всей формы). Шрифт внутри – тот же, что в основном тексте, размер 14–16px для удобства чтения. Placeholder – курсивом или светлым цветом, чтобы отличать от введенного текста. Метка поля (label) – располагается над полем или внутри (функционал "плавающего label": когда фокус – превращается в маленькую надпись сверху).
- **Состояния:** *Нормальное:* стандартная рамка, нейтральный фон. *Hover:* курсор текст (I-бим), можно немного оттенить фон (#f7f7f7) или рамку сделать темнее, чтобы поле при наведении выделялось (но не обязательно). *Фокус:* ярко выраженное состояние – рамка становится основного цвета (брендового), или появляется подсветка снизу толщиной 2px, фон может стать белым чище. Также метка (label) поднимается/становится активной (если реализовано). *Валидное:* (при желании) – зеленоватая рамка/иконка галочки. *Ошибка:* красная рамка или тень, сообщение об ошибке снизу красным мелким шрифтом. *Disabled:* более бледное, неактивное (фон серее, текст светлее, interactions off).

- **Анимации:** Основные – **плавный переход цвета рамки/фона** при фокусе (L1, 0.2s). Если используем плавающий label, его движение вверх и уменьшение размера – анимировать (L2, ~0.2s ease) для аккуратности. Появление сообщения об ошибке – fade/slide вниз (L2, 0.3s, чтобы внезапно не прыгало). В остальном, поля не должны анимироваться сильно, т.к. пользователь печатает и не должен отвлекаться. Можно добавить едва заметный **пульс курсора ввода** (стандартный мигающий каретку оставляем). При автозаполнении браузером – можем подсветить поле легкой анимацией фона (желтый flash на 1с, как часто делают).
- **Доступность:** Каждое поле связано с `<label>` (либо явным `<label for>` либо через ARIA). Для input ошибок – `aria-invalid="true"` и связка с элементом ошибки через `aria-describedby`. Фокус-стиль видимый (браузерный или кастом). Размер поля достаточен для шрифта + отступы (padding ~8px). Если поле имеет ограничения (мин/макс длина, формат) – использовать HTML5 атрибуты (`type`, `pattern`, `required`) и/или aria-hints. Обязательно обеспечиваем доступность выпадающих списков, чекбоксов, радио – либо нативные элементы стилизовать, либо если кастом – добавить role="checkbox" и keyboard controls. Плейсхолдеры не используют как единственный способ пометить поле (чтобы при вводе не терялась подпись). Цвет placeholder достаточно контрастен? Обычно #aaa на белом – может быть ок, главное, чтобы прочитали.
- **Производительность:** Обычные input – недорогие. Но если много кастом логики (маски ввода, сложные скрипты валидации), надо оптимизировать. Стараемся использовать **CSS вместо JS там, где можно** (например, CSS :valid/:invalid для простых валидаций). Избегаем тяжелых JS при каждом keypress, лучше делать дебаунс для проверки. Если есть live-подсказки по вводу (например, показывать превью при вводе ссылок), то грузить их по необходимости. Графические эффекты (тени, SVG иконки внутри поля) – минимально и кэшировать. Не делаем анимацию градиентов на полях или что-то, что тормозило бы при вводе.

Карточки (Cards)

- **Стиль:** Карточки используются для отображения самостоятельных блоков информации – например, товара, кейса, уведомления. Стиль карт – **единый модуль с собственной рамкой или фоном**, отделенный тенями или отступами от других. Обычно белый или слегка прозрачный фон на контрастном фоне сайта. Углы скругленные (как у кнопок, 4–6px для дружелюбности, или больше если нужен эффект «панели»). Каждая карточка содержит: изображение (опционально), заголовок, текст, возможно кнопку или метки. Пример: в галерее работ – карточка = фото + название работы; в профиле – карточка заказа = номер, статус, сумма.
- **Сетка и отступы:** Внутри карточки – поля контента отделены равномерными отступами. Внешние отступы между карточками одинаковые, чтобы сетка смотрелась ровно. При разных размерах карточек (например, Masonry плитка) – следим за выравниванием по сетке колонок.
- **Состояния:** Карточка сама по себе может быть кликабельна (например, вся карточка – ссылка). *Hover:* если кликабельна, вся карточка слегка приподнимается (тень усиливается, или трансформация Y=-2px) и курсор pointer. Если не кликабельна, то hover может отсутствовать или только для внутренних кнопок. *Active:* (при нажатии) – карточка слегка «проминается» (тень убирается, как кнопка). *Выбранная/выделенная:* если карточки могут выбираться (напр. выбор товара) – для выбранной меняется рамка на активный цвет или фон отличается.
- **Анимации: L1–L2 эффекты**. Hover подъем – transition-shadow, transform (0.3s). Появление карточек на странице – можно анимировать пачкой: напр., при загрузке галереи карточки появляются с fade-in + небольшим сдвигом вверх (L2, 0.4s, с лагом между элементами 0.1s

для каскада). Если карточек много, можно ограничиться появлением первой десятки, остальные просто появляются без анимации, чтобы не перегружать. Если карточка содержит изображение, при hover можно анимировать только картинку внутри: например, **эффект параллакса внутри карточки** – картинка чуть смещается относительно рамки (L2, зрительно объем). Либо затемнение изображения и появление текста поверх (например, в кейсах: при наведении на фото – затемняется фон и проявляется название и иконка «лупа»). Это делается через псевдоэлементы и transition opacity (легко).
- **Доступность:** Если карточка – ссылка, обеспечить атрибут `tabindex` и фокус-стили (например, при фокусе клавиатурой добавляем обводку вокруг карточки). Структура внутри должна быть семантична (заголовок `<h3>` например, описание `<p>`, и т.д.). Для screen reader карточка-ссылка должна иметь понятный текст (например, aria-label="Открыть кейс X"). Если карточки группируются, можно использовать список (`<ul><li>`). Текст внутри карточки не должен быть слишком мелким – минимум 14px. Цвета фона/текста – проверяем контраст (если фон карточки цветной, чтобы текст читался).
- **Производительность:** Если карточек много (сотни), нужно оптимизировать рендеринг: напр., lazy-load изображений (не загружать картинки вне экрана). Можно использовать Intersection Observer для подгрузки картинок при скролле. Анимацию появления делать только для первых элементов, или очень упрощенную для остальных. Тени карточек могут влиять на перформанс при скролле (особенно на мобилках), поэтому тень делаем не слишком размытой или используем `will-change: transform` на hover чтобы вынести на композитный слой. Также избегаем вложенных анимаций: если внутри карточки что-то двигается и сама карточка двигается – это двойная нагрузка. Лучше по минимуму.

Модальные окна (Modals)

- **Стиль:** Модалки появляются для подтверждения действий, отображения крупных превью (например, та же **линза сравнения** может быть в модалке), или для ввода доп. информации. Стиль модального окна: **центрированный контейнер с контентом**, фон затемняется. Фоновый оверлей – полупрозрачный чёрный (~50% opacity) либо размытие (эфект «стекло» – см. визуальные материалы) для затемнения остального интерфейса. Само окно – светлое или темное, зависит от темы, с скругленными углами (чуть больше, например 8px, чтобы показать, что это над карточками). Размер модалки – адаптивный: на десктоп ~50% ширины экрана или фиксированный, на мобилке во весь экран.
- **Содержание:** Внутри модалки располагаются: заголовок (например, “Подтвердите заказ”), текст/контент (описание, изображение и т.п.) и кнопки действий (OK/Cancel). Раскладка вертикальная. Кнопка закрытия (крестик) в правом верхнем углу модалки всегда, кроме простых алертов где только OK.
- **Состояния:** *Открыто/закрыто* – основные состояния. При открытии модал получает фокус (и фокус ловится на элемент внутри модалки). При закрытии – модал убирается из DOM. Состояние фоновых элементов – inert (отключены) пока модал открыт, чтобы нельзя было табнуться за пределы.
- **Анимации: Появление модалки:** мягкое, L2. Обычно: фон затемняется (fade-in, 0.3s), сама модалка всплывает (можно слегка масштабируясь от 0.95 до 1 и с fade). Можно добавить легкий overshoot: например, она чуть вылетает и отскакивает обратно (но очень деликатно, чтобы не комично выглядело). Закрытие – то же в обратном порядке (fade-out). Если используется View Transition API или другой метод, можно анимировать модалку более плавно. Но избегать долгих/сложных эффектов, модалка – утилитарна. **Внутри модалки** тоже можно анимировать: например, если это галерея-превью, внутри могут листаться изображения – это допускается, но как часть контента.

- **Доступность:** Модальное окно должно получать `role="dialog"` или `aria-modal="true"`, иметь обозначение заголовка (`aria-labelledby`) и описания (`aria-describedby`) при наличии. Фокус-трап: цикл фокуса внутри модалки, ESC – закрывает (вешается обработчик). Обязательно обеспечить, чтобы модал можно было закрыть без мыши (кнопка закрытия доступна для фокуса + ESC). Чтение экранными дикторами: при открытии модалки желательно добавить в DOM после body (чтобы читалась последней) или управлять focus. Фоновые элементы должны быть недоступны (можно wrapper с `aria-hidden="true"` для основного контента при показе модалки).
- **Производительность:** Модалки обычно редко используются, но нужно учитывать: если модальных окон много (например, галерея со 100 карточками и для каждой своя модалка в DOM), лучше генерировать их динамически по запросу, а не держать все. Анимация появления/скрытия – CSS, довольно безобидно. Но внимание к **backdrop-filter**: если используем размытие фона, на больших экранах это может нагружать, т.к. вся страница блюрится под модалкой. Возможно, для performance, делаем простой полупрозрачный фон без blur на мобильных (fallback). Также, отключаем любые фоновые видео/анимации, когда модалка открыта (например, если на главной шёл фон-видос, при открытии модалки его стоит паузить). Это снизит нагрузку на CPU/GPU, пока пользователь взаимодействует с диалогом.

Инструмент сравнения (Линза, Compare/Lens)

- **Описание:** «Линза» сравнения – UI-компонент для **сопоставления двух изображений или состояний**. Чаще всего, одно изображение накладывается на другое, и пользователь может перемещать разделитель (круглую “лупу” или вертикальную черту) чтобы видеть разницу. В нашем случае это применяется для: сравнить **оригинал vs. симуляция печати** (подложка, цвета), либо **до/после** какой-то обработки. Компонент состоит из двух слоев: нижний – изображение A, верхний – изображение B, и управляющий элемент (ползунок или кружок-линза).
- **Внешний вид:** Если **режим линзы (кружок)** – то при активации появляется круглая область, которую можно таскать. Внутри круга показывается изображение B, снаружи – A (или наоборот). Границы круга четкие, с тонкой обводкой, чтобы видно зону. Диаметр круга может быть ~150-200px (на десктопе), на мобилке чуть меньше относительно экрана, и адаптивный. Если **режим ползунка** – поверх изображения идет вертикальная (или горизонтальная) полоса-разделитель с ручкой, которую можно двигать, слева одно изображение, справа другое. Ручка может выглядеть как кружок/стрелка, подсвеченная при hover.
- **Состояния и взаимодействие:** Компонент активируется либо по клику на специальную кнопку («Сравнить»), либо присутствует сразу. *Default:* показывается одно изображение полностью. *On activate:* появляется второй слой и контроль. *Hover:* курсор меняется на перемещение (например, `cursor: grab`), элемент управления подсвечивается. *Dragging:* курсор `grabbing`, линза/ползунок следует за курсором или пальцем. *Release:* останавливается. Если кликнуть вне линзы (для режима линзы) – возможно, компонент закрывается (в зависимости от реализации).
- **Анимации:** Само перемещение линзы – не анимация, а непосредственный отклик на действия пользователя (следует под мышью/пальцем, без лагов, L0 – прямое). Но **появление и исчезновение** линзы/режима сравнения можно анимировать: например, плавное выведение второго изображения из прозрачности при активации (L2, 0.3s fade-in). Если используется ползунок, при активации он может слегка выплыть из края. Также можно добавить **индикаторное мигание**: когда пользователь включил режим сравнения, сам контрол (кнопка/значок) может мигнуть или покачаться (L2) один раз, указывая

"возьми меня и двигай". После этого никаких автодвижений – только пользователь. При деактивации – второй слой плавно исчезает.
- **Доступность:** В идеале, инструмент сравнения должен быть управляем и с клавиатуры (но это сложно UX-но). Минимум – предусмотреть альтернативу: например, кнопка «До / После» которая переключает изображения для тех, кто не может драга использовать. Либо клавиши стрелка влево/вправо двигают ползунок, если фокус на компоненте. Нужно озвучить для скринридеров: например, при активном компоненте давать текстовое описание "Изображение X сравнивается с Y. Нажмите кнопку переключения режимов для просмотра.". Линза как чисто визуальный инструмент может быть скрыта от доступности, а вместо него – две картинки и описание разницы текстом (для людей с плохим зрением). Цвета и границы линзы – достаточный контраст с фоном изображения, чтобы видно было раздел (на темном изображении – белая окантовка, на светлом – тень).
- **Производительность:** Накладывать два больших изображения друг на друга – потенциально тяжело для памяти, но в целом просто для рендера (два <img> stacked). Однако перемещение линзы требует, чтобы браузер быстро обновлял маску/клип. Решение: использовать CSS `clip-path` или `<svg><mask>` для круглой линзы – эти операции выполняются на GPU и довольно быстрые, особенно если использовать `will-change: clip-path` на элементе с маской. Для ползунка – можно просто изменять ширину одного из div с overflow:hidden – тоже быстро. **Важно:** отключить тяжелые эффекты на изображениях: никаких CSS-фильтров на каждом кадре движения (т.е. если нужно фильтровать, применить заранее). Также убедиться, что при перемещении не происходит перекомпоновки layout (использовать absolute positioning для слоев). Если заметны просадки FPS при драге на слабых устройствах, можно: а) уменьшить частоту обновления (throttle `mousemove`), б) упростить маску (например, использовать квадратную маску вместо сложной). Но лучше стремиться к идеально плавному, особенно на десктопе (60fps). Для мобильных – протестировать на тач: если аппарат слабый, возможно, вместо свободной линзы дать только ползунок (проще вычислений).
- **Фоллбеки:** Если браузер не поддерживает нужные CSS (например, старый IE не знает clip-path) – можно тогда просто показывать два изображения рядом или одно под другим, сопровождая текстом «до/после». Это хуже UX, но хотя бы даст инфо. Codex-агент должен предусмотреть условие или CSS, чтобы degrade gracefully.

Таймлайны (Timelines)

- **Описание:** Таймлайн – компонент для отображения последовательности этапов или событий во времени. Применение: **статус заказа** (этапы от принятия до доставки), история операций, дорожная карта. Представляет собой вертикальную колонку или горизонтальную линию с узлами (nodes) – каждый узел = этап, подключен линией. Может включать текст (название этапа, время) и иконку статуса.
- **Стиль: Вертикальный таймлайн:** линия (2px толщина, например) вдоль левого края компонента, точки-узлы на линии. Активный/завершенный этап – отмечен залитой точкой или значком (галочка), будущие – пустые кружки. Цвет активного узла – брендовый акцент, завершенные – нейтральный или тоже акцент, но с галочкой внутри. Текущему этапу можно придать особый фон или выделить карточку. **Горизонтальный таймлайн:** например, 3-4 шага по горизонтали, соединены линией. Аналогично, текущий шаг выделен цветом и количеством заполнения линии до него. Шрифт в таймлайне мелкий-средний, чтобы влезало описание.
- **Состояния:** *Initial:* отображает до текущего этапа заполнено, остальное пусто. *Progress:* (если анимируем) – при переходе этапа, следующий узел меняет состояние. *Hover:* если узлы интерактивные (например, можно кликнуть на этап, чтоб подробности), тогда hover подсвечивает его, курсор pointer. Но если чисто информативный, то узлы не интерактивны.

Completed: стилистика завершенных этапов (галочки, заштрихованные). *Current:* особое оформление текущего – напр., мигающий индикатор или пульс вокруг точки (если хотим показать активность).

- **Анимации: Отрисовка таймлайна:** при загрузке страницы статуса можно анимировать **заполнение линии** до текущего этапа (L2): напр., линия растёт вертикально от первого до текущего узла, длительность 0.5s. Узлы могут появляться с небольшим скейлом (появился кружок и расширился до размера). При обновлении статуса в реальном времени: новый узел – **микро-анимация L2** (например, иконка галочки вспыхнула внутри кружка, или кружок сменил цвет с плавной заливкой). Текущий этап может иметь **пульсацию** (L1) – едва заметную, например обводка, чтобы привлечь внимание, что "сейчас этот процесс идет". Но не более того, чтобы не отвлекать постоянно. Если таймлайн кликабельный (например, показывает годы истории компании) – при клике можно прокручивать или выделять соответствующий контент; тогда анимация прокрутки мягкая.
- **Доступность:** В верстке таймлайн может быть списком (`<ul><li>`), что хорошо для скринридеров – каждый пункт читается как “этап 1: название, завершено” и т.п. Нужно добавить скрытые текстовые метки статусов, например, "завершено", "текущий этап". Цветовая индикация (зеленый/серый кружок) должна дублироваться текстом или значком (галочка vs пустой круг) – для людей с дальтонизмом. Если горизонтальный таймлайн, убедиться, что на мобильных он доступен (может стать вертикальным или скроллиться). Фокус: если узлы интерактивны, они должны быть достижимы табом и иметь aria-label типа "Этап 2: Печать завершена".
- **Производительность:** Таймлайн – довольно простой графически (SVG или div-ы), проблемы могут быть только при очень длинных списках. В нашем случае этапов немного (~5-7). Анимация линий и точек – через CSS трансформации или SVG stroke-dashoffset (оба варианта ок). Они не требовательны. Главное – не делать чрезмерно сложных теней/blur на фоне таймлайна, это ни к чему. Если обновление статуса происходит часто (polling), оптимизируем DOM-операции: обновляем только нужный узел (HTMX пригодится). Следим, чтобы анимация не запускалась слишком часто (например, если статусы меняются очень быстро – что маловероятно для заказа). В целом, таймлайн – легковесный компонент.

(Другие компоненты: **меню/навигация**, **таблицы**, **иконки** – оформляются по общим принципам: чистый дизайн, hover-фидбек, консистентность с остальными элементами. Например, навбар фиксированный сверху, полупрозрачный фон, ссылки с подсветкой при hover и индикатор текущей страницы.)*

<!-- Codex: Maintain consistent component styling. Use utility classes from a design system if available. Ensure each component's interactive states (hover, focus, disabled) and ARIA attributes as described. -->

5. Визуальные материалы и стиль

Эта секция описывает глобальные визуальные линии: цвета, типографику, текстуры и эффекты, которые формируют единый облик сайта. Здесь закладываются **дизайн-токены** и стилевые направляющие.

- **Цветовые токены:** Проект использует систему цветовых переменных для консистентности. Базовая палитра включает:
- `--color-bg` – цвет фона основной. Вероятно, **темный фон (#0F0F0F или глубокий графит)** для страниц в стиле «control deck», чтобы изображения и элементы ярко выделялись. Альтернативно, если решено пойти в светлую тему: `--color-bg` = белый/светло-серый, а блоки интерфейса темными. Но философия Lab Proof + Control Deck скорее

предполагает **контрастную схему** (темный UI, светлые содержимые элементы или наоборот). Это уточняется на этапе визуализации, но токен позволяет менять.

- `--color-text` – основной цвет текста. На темном фоне это **светлый (почти белый)** #F0F0F0. На светлом – наоборот темный #202020. У нас должно быть высокое контрастное сочетание с `--color-bg` (контраст > 11 для главного текста).
- `--color-primary` – **акцентный цвет бренда**. Используется для важных элементов (кнопки, ссылки, иконки состояний). Должен отражать стиль компании: для технологичного, экспериментального духа подходят **яркие неоновые оттенки** (напр., неоново-зелёный #00FFA2, электрический синий #00AEFF, или яркий оранжево-янтарный). Выбор конкретного оттенка согласуется с брендбуком TwoComms, если он есть. Этот цвет применяется в контролах (кнопки, галочки, акценты) и в эффектах (подсветка активного).
- `--color-secondary` – вспомогательный акцент. Если нужно второй цвет (например, для выделения другого типа информации, вторичных кнопок), можно ввести. Часто это более мягкий, например, серо-синий или фиолетовый, дополняющий primary. Может отражать аспект «лабораторности» – например, мягкий пурпурный как в подсветке приборов.
- `--color-success`, `--color-error`, `--color-warning` – служебные цвета для статусов (успех – зелёный, ошибка – красный, предупреждение – желтый/оранжевый). Используются в тексте уведомлений, обводках полей и т.д. Они должны вписываться: например, зеленый может быть таким же, как primary если brand = green, либо отдельным.
- `--color-bg-alt` и `--color-surface` – альтернативные цвета подложек. Например, если фон страницы темный, то всплывающие панели (surface) – чуть светлее/темнее для контраста. Noise-слой может накладываться на них (см. ниже).
- `--color-border` – цвет границ между элементами (скажем, #2A2A2A для темной темы).

- `--opacity-overlay` – уровень прозрачности для оверлеев (например, модальное затемнение), можно вынести как токен (например, 0.6).
  Все цвета задаются в одном месте (CSS variables) для удобства смены темы. Возможно, будет два режима: **Dark** и **Light** theme – с разным набором выше токенов, чтобы при необходимости переключить (Control Deck vibe может быть dark mode by default).
  Контрастность: Проверяем сочетания: основной текст vs фон >= 4.5:1, мелкий вспомогательный текст >= 3:1. Акцентные элементы на фоне тоже должны быть различимы. Цветовые токены лучше выбрать так, чтобы даже при конверсии в черно-белое интерфейс был понятен (на случай монохромного принтера или дальтоников).

- **Шрифты и типографика:** Шрифтовая пара для проекта должна сочетать **технологичность и читабельность**. Предлагается:

- **Заголовки:** Использовать **display-шрифт** с характером. Возможен вариант: геометрический **без засечек** (например, *Montserrat*, *IBM Plex Sans*, *Inter*, *Oswald* – в зависимости от желаемого настроения). Он должен выглядеть современно и немного “технически”. Если бренд TwoComms имеет фирменный шрифт – используем его. В случае отсутствия – можно взять Google Fonts с лицензией. Важно: шрифт поддерживает кириллицу/латиницу. Заголовки могут быть в **uppercase** для брутальности, либо обычным регистром. Размеры: H1 ~ 36px, H2 ~ 28px, H3 ~ 22px, H4 ~ 18px, далее текст. Заголовки отделены хорошим пространством сверху/снизу (типографический ритм).
- **Основной текст:** Простой **рубленный (sans-serif)** шрифт высокого удобочтения. Например, *Roboto*, *Arial* (fallback), *Nunito* или тот же *Inter* (если используем его и для заголовков, но разной насыщенности). Кегль базовый ~16px на десктопе (на мобильных 14-16px). Межстрочный интервал ~1.5 для комфортного чтения параграфов. Цвет текста – см. `--color-text`.

- **Моноширинный шрифт:** Может понадобиться для отображения кодов заказов, технических деталей (как элемент стиля “Lab Proof”, показать артикул как на наклейке). Например, *Roboto Mono* или *Courier New* для спец. элементов. Использовать спорадически, чтобы выделить кусок информации (например, номер заказа #A001 как на бирке).
- **Вес и насыщенность:** Желательно использовать варирующиеся начертания: Light, Regular, Medium, SemiBold, Bold. Например, заголовки H1 – Bold 700, подзаголовки – Medium 500, текст – Regular 400, кнопки – Medium 500 (лучше чуть более жирные, чтобы не терялись).
- **Фирменные акценты:** Возможно, часть текста может оформляться уникально, например цифры в номерах – другим цветом или шрифтом, как элемент стиля. Но аккуратно, чтобы не превратилось в noise. Если есть логотип-гарнитура, её можно применять в крупных надписях (например, слово "TwoComms" особым шрифтом).
- **Типографические принципы:** Выравнивание текста – в основном по левому краю (для удобства чтения). Центрирование – только для заголовков на главной, коротких строк. Не использовать **длинные строки** текста – ширина контейнера для параграфов ~60–80 символов (потому что иначе тяжело читать). Следить за висячими предлогами (в русском) – опционально, можно мягкий перенос или неразрывные пробелы, Codex-агент должен обращать внимание. При уменьшении экрана – шрифт может чуть уменьшаться, но стараться сохранять читабельность.

- **Иконки:** Они тоже часть типографики. Можно использовать иконочный шрифт или SVG-иконки. Стиль иконок: тонкие линейные (line icons) или простые геометрические – чтобы сочетались со шрифтом. Размер иконок в тексте ~1em по высоте, выравнивать по базовой линии текста.

- **Шумовые слои (noise), текстуры:** Чтобы интерфейс не выглядел слишком плоским и цифровым, вводятся **тонкие шумовые текстуры**. Это отражает философию тактильности (микро-тактильность) и лабораторной реалистичности (как будто материалы настоящие). Реализация: поверх однотонных заливок (фонов) накладывается полупрозрачный шум (зернистость) очень низкой интенсивности (например, 5-8% opacity). Можно использовать прозрачное PNG 10x10px с точками или генерировать через CSS (например, `background: url('noise.png'), linear-gradient(...)`). Шум добавляем на большие однотонные области: фон страницы, фон хедера/футера, фон карточек, модалок. Он практически не виден на первый взгляд, но убирает идеальную гладкость – глаз чувствует структуру. В стиле Lab Proof это придает реализм, будто UI отпечатан на физическом носителе или глядим через прибор. **Важно:** шум должен быть **статичным** (не анимированным) и очень мелким, чтобы не отвлекать и не портить читаемость. Например, черно-белый шум с альфа 0.03. Можно вариации: на темных областях – светлый шум, на светлых – темный (две текстуры).

- **Градиенты и фоны:** В целом дизайн скорее плоский, но можно применять **градиенты** аккуратно – например, для акцентной кнопки (оттенок primary чуть темнее к чуть светлее, чтобы объем придать). Либо для фона страницы – темно-серый к чуть менее темно-серому, чтобы создать глубину. Градиент должен быть **нейтральным**, без радужных переливов, если только это не фирменный стиль. Возможно, фон hero-блока – легкий градиент + шум + еле заметная анимация света (например, проходящая тень от движущейся принт-головки).

- **Имитация материалов:** Подход Lab Proof может включать эффекты, имитирующие реальные материалы: стекло (см. далее), пластик панели, свет от индикаторов. Если, например, хотим оформить какую-то панель как **экран осциллографа**, можно слегка

позеленить и добавить скан-линИИ, но это скорее концепты – если использовать, то очень умеренно и для декоративного блока, не для основного UI.

- **Стеклянные эффекты (glassmorphism):** Чтобы подчеркнуть high-tech стиль, уместно использовать **полупрозрачные стеклянные панели**. Например, навигационная панель или всплывающие подсказки могут иметь фон `rgba(255,255,255,0.1)` + `backdrop-filter: blur(10px)`, создавая ощущение матового стекла. Это добавляет глубины: элементы будто находятся в стеклянных контейнерах. В сочетании с шумом (добавить легкий шум на саму панель) получается правдоподобный эффект. Применение: фон модального затемнения – размытие, карточки UI – слегка прозрачные если накладываются на фон (например, уведомление поверх контента – как стеклянный прямоугольник). **Ограничение:** стеклянные эффекты дороги для GPU, поэтому:

- Использовать их **точечно**: либо верхний навбар (тогда ниже контент размывается при скролле под ним), либо отдельные виджеты. Не делать каждый блок полупрозрачным.
- **Fallback:** Если браузер не поддерживает backdrop-filter (например, Safari старых версий без флагов, или Firefox до недавнего времени), то вместо прозрачного стекла использовать однотонный, но полупрозрачный без blur, или просто солид цвет с пониженной непрозрачностью (тем самым текст сзади будет просвечивать, но четко, без blur).
- Проверить **контраст на стеклянных фонах**: текст на них, учитывая просвечивающее, может терять контраст. Обычно решают добавлением чуть более темного фонового цвета под текст.
- Стеклоэффект лучше смотрится на **темном UI** с цветными бликами. Если общий фон светлый, стеклянный эффект менее заметен (белое на белом). Но можно в светлой теме использовать белое стекло с тенью.
- Производительноть: backdrop-filter применяем на относительно небольших областях (на весь экран – риск фризов на слабых). Например, навбар 60px высоты – нормально; модалка на весь экран с blur – может лагать на старом телефоне, поэтому либо отключить blur на мобиле, либо сделать blur меньше радиуса.

- **Примеры использования:** плашка с уведомлением «Заказ добавлен» – появляется как полупрозрачная полоска поверх контента. Главное, не злоупотреблять – 1-2 разных места со “стеклом” достаточно, иначе потеряет новизну эффекта.

- **Микро-тактильность:** Под этим подразумевается, что пользователь **чувствует отклик интерфейса словно от физического объекта**. Визуально и интерактивно мы даем микро-сигналы, имитирующие реальную тактильность:

- **Тени и слои:** Элементы расположены слоями с тенями, что дает ощущение глубины – как кнопка над панелью. При нажатии кнопка “уходит” ближе к панели (тень уменьшается). Карточки при перетаскивании (если такое предусмотрено) отбрасывают тень на поверхность, как реальные бумажки.
- **Анимация нажатия:** Как описано, при клике элементы чуть уменьшаются – мозг воспринимает это как нажать реальную кнопку. В сочетании со звуком (опционально) или вибрацией (для мобилок есть `navigator.vibrate()` – можем использовать на особо важных нажатиях, но осторожно, не злоупотреблять) создается полный тактильный отклик.
- **Микро-звуки:** (Хотя не упомянуто явно, возможно допускается) – например, тихий звук «щелчка» или «писка сканера» при определенных действиях. Если внедрять, то предоставлять опцию выключить. Но Codex вряд ли будет генерировать звуки, это больше к контенту.

- **Курсоры и подбор элементов:** Наведение курсора меняет форму (pointer, text, move) – это тоже микросигнал. Можно кастомизировать курсор слегка (например, при наведении на Printhead Scan эффект – курсор становится иконкой принтера).
- **Физические эффекты:** Если допустимо в вау-контексте: мелкие **пружинные анимации** (spring physics) на меню или переключателях, чтобы ощущалось как реальные переключатели. Например, тумблер “ON/OFF” дергается с легким пружинным эффектом при переключении – как будто физический. Но такие детали – приоритет низкий, только если время позволяет.
- **Отзывчивость интерфейса:** К микротактильности можно отнести и скорость реакции – интерфейс сразу откликается на действия (например, кнопка мгновенно подсвечивается на наведение, без задержек). Это важно, чтобы пользователь чувствовал “связь” с UI.
- **Микро-анимации контента:** Например, количество в корзине увеличилось – цифра слегка подпрыгнула. Или пришло новое уведомление – значок конверта дрогнул. Эти маленькие жесты делают интерфейс “живым” и приятным.

Вся визуальная система (цвет, шрифт, шум, стекло, тактильность) должна работать в комплексе: *технологичность + премиальность + удобство*. Пользователь может и не заметить сознательно шум или стекло, но подсознательно интерфейс будет ощущаться более богатым и внимательным к деталям. При этом **никакой из визуальных приемов не должен ухудшать чтение или скорость работы** – баланс красоты и функциональности.

<!-- Codex: Use CSS variables for colors and fonts. Ensure all visuals degrade gracefully on older browsers (e.g., no backdrop-filter -> use solid colors). Keep a consistent style through all components as defined. -->

6. Техническая реализация эффектов

В этом разделе описаны ключевые технологические моменты реализации WOW-эффектов и интерактивности, с акцентом на производительность. Мы используем современные веб-API и подходы: **View Transitions API** для плавных переходов, библиотеку **HTMX** для частичной загрузки контента без перезагрузки, и разрабатываем собственные функции (Printhead Scan, Underbase Simulation, Paste-to-Order, Compare/Lens).

Эффект Printhead Scan (фирменный WOW-эффект)

- **Идея и назначение:** Printhead Scan – анимация, имитирующая работу печатающей головки принтера DTF. В интерфейсе это проявляется как **движущаяся горизонтальная (или вертикальная) полоса** по изображению, которая постепенно “проявляет” его, словно печатая строчку за строчкой. Это не только зрелищно, но и обучающе – пользователь видит технологию в действии. Планируем задействовать этот эффект на главной странице (для вау-презентации) и/или на Preflight (в качестве функциональной проверки, как печататься будет макет).
- **Уровни реализации (Motion tiers):** Эффект будет адаптивным:
- *L4 (максимум):* Полноценная симуляция печати. Головка (графическое представление – например, тонкая прямоугольная “линия” с имитацией сопел или лазерного свечения) проходит по изображению **несколько раз**, постепенно наращивая цвет. Можно сделать 2–3 прохода разными цветами (CMYK), если это впечатлит, или один проход, но с очень высокими деталями (например, искры или легкий дымок – только если это легко). В процессе прохождения может сопровождаться тихим звуком печати (шум механики – опционально). В конце – полностью отпечатанное изображение. Примерная длительность L4 версии: 2–3 секунды на один проход (если несколько – до 5–6с суммарно). L4

используется на десктопе, когда мы уверены, что пользователь готов посмотреть демонстрацию (на главной в hero или в Preflight по запросу).

- *L3 (стандарт):* Упрощенная, но все еще эффектная версия. Один проход сканирующей полосы. Полоса может быть с небольшим свечением, она двигается сверху вниз (или слева направо) за ~2с, сразу раскрывая изображение полностью (например, за полосой картинка уже "напечатана"). Возможно, в начале изображение полностью прозрачное/бледное, а полоса как бы открывает его. Т.е. реализация через CSS mask: полупрозрачная белая полоса с градиентными краями, которая движется, а за ней изображение меняет opacity с 0 до 100%. Эта версия гораздо проще технически и легче для GPU. L3 вариант – основной, который пойдет большинству пользователей.
- *L2 (минимум):* Еще упрощеннее: не анимация проявления, а просто **визуальный скан-эффект** поверх изображения. Например, когда пользователь наводит курсор на превью, появляется тонкая линия, быстро пробегает и исчезает, *не влияя на изображение*. Это чисто декоративно – показать "лазер". Реализация: CSS анимированный градиент или `<div>` 1px height, animating position. Может повториться 1-2 раза. Изображение само уже видно изначально. Такой эффект легкий и быстрый (<<1с), можно использовать на мобильных или если полная анимация отключена.
- *L1:* Может сводиться просто к подсветке изображения рамкой или тенью (т.е. практически без движения) – если даже L2 нежелателен.
- *L0:* Полный отказ от эффекта (статичное изображение) – при отключенных анимациях или слабом окружении.
- **Реализация фронтенд:**
- Используем сочетание CSS и JS (или Web Animations API). Оптимально: иметь изображение в контейнере с `overflow: hidden`. Поверх – элемент "scanner" (полоса). Если делаем проявление, можно два canvases: один – исходное изображение, второй – маска, которая увеличивается. Но можно проще: CSS `clip-path: inset()` анимировать. Например, начально изображение имеет clip-path, скрывающий его (0% видимости по вертикали), потом по мере движения увеличиваем видимую область.
- Для более реалистичного эффекта, можно написать на Canvas: iterate over image rows, reveal gradually. Но это потребует JS loop, может лагать на больших изображениях. Лучше пользоваться **CSS transitions** – они хорошо оптимизированы. Например, `img { clip-path: inset(100% 0 0 0); transition: clip-path 2s linear; }` и потом установить clip-path: inset(0 0 0 0) – изображение откроется сверху вниз. Параллельно можно сверху пустить полоску: `<div class="scanner-line">`, позиционировать по Y (или X) соотв. концу видимой части. Даже можно синхронизировать: CSS `@keyframes` движущейся полосы от top:0% до top:100% за 2s linear, и анимация clip-path с тем же временем. Чтобы синхронизировать, можно либо разделить на steps, либо использовать `animation-play-state`/delay.
- Полоса может быть стилизована: градиент (белый полупрозрачный центр, прозрачные края) + box-shadow for glow. Если хотим отображать сопла, можно на полосе мелкие кружочки. Но это мелочи дизайна, главное – движущаяся линия.
- **Запуск:** эффект запускается когда элемент в зоне видимости (для главной – сразу на загрузке hero, или когда секция прокручена), для Preflight – при клике "Просканировать" или автоматом после генерации превью. Используем `IntersectionObserver` или события. **Повтор:** Обычно эффект один раз проигрывается. Если нужно повторить, например, по нажатию кнопки "Еще раз", мы должны сбросить состояние (например, re-trigger CSS animation by removing and re-adding class).
- **Остановка/Пропуск:** Пользователю дать возможность прервать: напр., на Preflight кнопка "Пропустить анимацию" – тогда мы мгновенно показываем финальное изображение (убираем clip-path, скрываем полосу).

- **Performance:** На больших изображениях (например, 4000px высоты) анимация clip-path может кушать ресурсы. Лучше ограничить макс-размер превью, например, отображать уменьшенную версию. Также, если делаем Canvas pixel-by-pixel – это точно избегаем, только CSS/GPU. Animating clip-path in Chrome is hardware accelerated (especially if using polygon or inset), but for safety, consider using transforms: e.g., have two copies of image – one masked one unmasked – that might complicate. The CSS approach described is fine. **Testing on mobile:** ensure 2s of clip animation doesn't cause frame drops – likely fine on modern phones, but older ones? We might degrade to the L2 effect (no heavy clip, just overlay line) on devices below certain screen width or if known performance issues.
- **Coordination с ViewTransitions:** If page transitions happen, ensure that Printhead animation finishes or is canceled properly to not carry over. But likely not an issue if it's contained.
- **Прочее:** Document this well in code comments so developers understand the effect. Possibly allow some customization via CSS variables (speed, color of line etc). Codex-агенту: нужно генерировать анимации аккуратно, test cross-browser (Chrome has full support, Safari tech preview only might need flags for view transitions but for basic CSS it's fine).

Анимация переходов страниц (View Transitions API)

- **Назначение:** Сделать переходы между страницами (или крупными разделами) плавными, без резкого “перескакивания” контента. С View Transition API браузер сам анимирует разницу между старым и новым DOM, что создает приятный эффект целостности приложения, приближая нас к SPA-опыту но без потери MPA простоты. Мы хотим использовать этот API для основных навигационных переходов: например, при переходе с Главной на Галерею – геро-изображение плавно трансформируется, заголовок перемещается, или просто общий fade (в зависимости от разметки).
- **Использование API:**
- В целях максимальной простоты можно задействовать **CSS декларативно**: добавить в глобальные стили `@view-transition { transition: transform 0.4s, opacity 0.4s; }` или `:root { view-transition-name: page; }`. Также пометить элементы, которые мы хотим анимировать между страницами, атрибутами `view-transition-name`. Например, на обеих страницах логотип в шапке можно назвать `view-transition-name: logo` – тогда при переходе логотип не исчезнет резко, а плавно переместится/изменится между состояниями. То же может быть для фона страницы, etc.
- Более контролируемый способ – JS: `document.startViewTransition(() => { window.location.href = ... })` или (если HTMX usage complicates, then for full navigation we could rely on browser if enabled via `navigation: auto`). Chrome поддерживает `<navigation-transition>` in CSS if you set it. Actually as of 2024, to animate full page nav, need to set `document.documentElement.style.viewTransitionName = 'something'` on both pages and have `navigation: same-origin` flagged in CSS. The specifics:
    - We can try using `@view-transition { navigation: auto; }` which automatically transitions on same-origin nav (Chrome supports via flag, soon stable). This covers any link click automatically. However, we must be careful: RUM data shows it might add ~70ms to LCP on mobile [5], so we use selectively.
- Likely approach: On desktop, enable `navigation: auto` (or do manual transitions for specific links), on mobile low-end either do no transitions or simpler ones. This “Selective Implementation” aligns with recommendations [6] – сначала тестируем, где transitions дают пользу, и где нет (возможно, на очень быстрых navigations user barely notices).
- **Переходы, которые анимируем:**
- Основная навигация (Home -> Order, Order -> Status, etc) – лучше **не анимировать** между совершенно разными контентами, т.к. может быть странно. Например, переход с лендинга

на форму – можно просто сделать fade or slide of whole page, or none (since focus should quickly move to form).

- Где анимация уместна: Главная -> Галерея (можно сделать красивый переход: последний секция Главной с превью галереи перетекает в Галерею целиком); Галерея -> детальный просмотр работы (если есть страница детали – можно переносить изображение).
- Order -> Preflight -> Status – можно анимировать **progress**: напр., на Preflight кнопка “Заказать” -> при клике происходит переход на Status, можно захватить, например, превью изображения, чтобы оно полетело в угол, но это усложняет, может без этого.
- Внутри одной страницы: например, пошаговая форма Order (если не делаем отдельные HTML, а просто скрываем/показываем секции) – тут ViewTransition API тоже поможет: `startViewTransition()` вокруг DOM изменений (HTMX swap or manual class toggles) для анимации изменения.
- **Примеры анимаций:**
- **Fade between pages:** Easiest global effect: old page fades out as new fades in. The API can do this if no common elements named, just via `@view-transition` specifying default. Good fallback if nothing custom.
- **Shared element transitions:** If we tag, say, the page background or a hero image or a title, they will smoothly morph. Eg: Home hero image transitions to Gallery page header image (if they share same element but that requires them to actually be the same resource or at least same size to morph nicely). Possibly we won’t have identical elements. But maybe brand logo in header is constant – we can tag that so that the header doesn’t flicker, it slides or stays put.
- **Slide transitions:** We might simulate slide by initial page position off-screen. But viewTransition might not directly do sliding of whole page easily (since it’s mainly good for shared elements or cross-fades). Might be easier to manually add a class and CSS translate for new page from right to left, etc. But that reintroduces custom code.
- Given complexity, likely we do mainly fades and a couple of shared elements (like persistent nav).
- **Performance considerations:** As noted, on slow devices the overhead of these transitions can hurt metrics (LCP). A measured ~70ms delay for repeat navigations on mobile was found [5]. So:
- Possibly wrap in a media query: `@media (min-width: 768px) { @view-transition { navigation: auto; } }` – meaning only desktops and big tablets do transitions automatically [7].
- Or detect device capabilities (maybe userAgent check for low-end Android to disable).
- Or allow user to opt-out (if they enabled reduce-motion, we definitely disable transitions).
- We'll A/B test enabling it to ensure it doesn’t tank conversion or metrics significantly (see Metrics section).
- **Fallback:** If browser doesn't support the API (only Chrome 107+ at 2024, Safari 16.4+, no IE obviously), then navigation will just be normal (no harm done). For critical transitions we want even on unsupported – could implement a manual CSS solution: e.g. add class to <body> on link click to do fade out, then new page loads normally (some flash though). Perhaps not needed. Focus on supporting progressive enhancement.
- **Integration с HTMX:** HTMX might intercept links and do internal swaps. If we want view transitions on those content swaps, we can manually call `startViewTransition`. Eg: when HTMX loads new content for a div, wrap it:

```
hx.on('htmx:beforeSwap', (evt) => {
  evt.detail.shouldSwap = false; // prevent default swap
  document.startViewTransition(() => {
    evt.detail.target.innerHTML = evt.detail.xhr.responseText;
  });
});
```

This way, the content change is animated. Codex should implement such if needed and if browser supports. Otherwise, just do swap.
- Summation: Use view transitions for an extra layer of polish on supported devices, but ensure site works fine without. Document clearly where applied.

Частичные обновления и динамика с HTMX

- **Назначение:** HTMX позволит нам делать интерактивность на сайте без полноценной SPA. Мы будем загружать фрагменты HTML динамически, реагировать на действия пользователя (клики, формы) и даже **периодически обновлять части страницы** (polling), без перезагрузки всей страницы. Это улучшит UX (быстрее реакции) и позволит плавно интегрировать анимации (с View Transition или CSS) при обновлении контента.
- **Примеры использования:**
- **Обновление статуса заказа (Status page):** Страница статуса может автоматически запрашивать обновленное состояние заказа каждые N секунд. HTMX поддерживает `hx-get` с `hx-trigger="every 5s"` и `hx-swap="outerHTML"` на контейнер статуса. Мы применим это к, например, элементу таймлайна: он будет обращаться к `/status/${orderId}` endpoint, возвращать обновленный HTML фрагмент таймлайна. HTMX подменит контент. Мы можем добавить `hx-indicator` для отображения индикатора загрузки (например, прелоадер) пока запрос идет, хотя 5s polling, обычно быстро. Важный момент: при swap нужно **сохранять прокрутку** (но на статусе статичная) и **анимировать изменения**. Как выше указано, можно обернуть HTMX swap в View Transition (conditionally, if supported). И/или применять CSS класс изменения (HTMX позволяет `hx-swap-oob`).
- **Форма заказа (Order page):** При смене опций, например, пользователь выбрал тип материала – можно динамически обновить цену или доступность. HTMX: `<select name="material" hx-get="/order/calc-price" hx-target="#price" hx-swap="innerHTML">`. Это без перезагрузки покажет новую цену. Быстро и удобно. Codex-agenty нужно учесть, что backend должен отдавать нужный фрагмент.
- **Preflight checks:** Когда пользователь загрузил файл и переходит к Preflight, возможно, мы отправляем файл на сервер для анализа (цветовой профиль, наличие альфа-канала и т.д.). HTMX может помочь: при загрузке файла (onchange в <input type=file>) – послать форму (hx-post) к /preflight, сервер вернет HTML: превью + список предупреждений. HTMX загрузит в целевой див. Так у нас бесшовно страница Order может переходить к Preflight секции без hard refresh.
    - Если же Preflight отдельная страница, можно всё же использовать HTMX: вместо полной перезагрузки при сабмите формы заказа – сделать hx-post на форму, сервер вернет Preflight HTML, swap whole <main>. To user, it feels like a step wizard.
    - Combine with view transition: while swapping main content, do a fade transition.
- **Галерея – подгрузка работ:** Галерея может иметь кнопку "Загрузить еще" или auto-infinite scroll. HTMX can fetch more items: e.g. `<div hx-get="/gallery/page2" hx-trigger="revealed" hx-swap="afterend">Загрузка...</div>`. When this placeholder scrolled into view, HTMX loads next batch and places after current items. That plus a fade-in CSS gives nice infinite scroll effect.
- **Lens compare modals:** If clicking a work opens a modal with compare lens, we can load that via HTMX (on click hx-get="/work/123/modal" hx-target="body" hx-swap="beforeend"). That returns a modal HTML. Then a bit of JS to show it (or CSS with backdrop). After done, remove it.
- **General navigation:** As a rule, we do **not** want to hijack all navigation with HTMX, because that essentially becomes an SPA but with heavy partial loads. There's an article "Less htmx is More" [8] cautioning not to overuse it. So:
    - Use HTMX where it significantly improves UX (forms, small content refresh).

    - For major page transitions (like going from Home to an entirely different section), might do normal link or anchor. Possibly a mixture: anchor triggers view transition, full page reload with nice effect is okay if caching is fine.
    - Or use HTMX for a small number of pages if beneficial. But converting entire site to one page might complicate state management (like title, meta, back button).
    - Likely, keep distinct pages for distinct tasks (the ones enumerated), use HTMX to enrich within those pages.
- **Progress indicators:** HTMX has `hx-indicator` attribute to show a spinner when request in progress. We should define a global spinner element (maybe top-right small loading bar or overlay). For example, a linear progress bar at top (like YouTube), that appears on any HTMX request. Or a small spinner next to the area updating (like next to "status: printing... spinner").
- Codex should implement CSS and small element for indicator, or use built-in `htmx-indicator` class toggling on target.
- **Push-state:** If using HTMX to swap content, we should manage history if needed (htmx supports `hx-push-url`). E.g., if Preflight loaded via HTMX, we want URL to update (so user can refresh or share link). Add `hx-push-url="true"` on that swap, so that new content gets its own URL in history.
- Similarly, for gallery infinite scroll, we might not push every page though to not flood history, but maybe push when user clicks explicit "load more" as a page param.
- **Security:** Ensure forms have proper CSRF tokens etc (not design's scope, but codex code generation should not break security). Possibly out of scope here.
- **Performance:** Partial updates are meant to be faster. But ensure we don't overload with too many requests (throttle polls, use condition on scroll triggers).
- Also, memory leaks: after repeated partial loads, ensure removed elements free up (HTMX usually replaces inner HTML which should garbage collect old).
- Evaluate if some heavy libraries or scripts need re-init after swap (HTMX triggers events `htmx:afterSwap` we can hook to re-init JS if needed).
- **Fallback:** If user has JS off, HTMX won't work, site should degrade to full page loads (forms have action URLs normally, links navigate normally). We ensure forms and links still have proper href/action so that even without AJAX the flow works (just less fancy).
- **Integration with other tech:** If using ViewTransition with HTMX, as described, need careful event handling to wrap content replacement. Alternatively, accept that partial content may not animate fancy but at least update quickly.
- The Codex agent should produce code that uses HTMX attributes and minimal custom JS. Complex logic (like an image processing or multi-step transitions) can be handled server-side or with minimal JS event hooks if needed. The aim is to avoid heavy front-end frameworks, and keep things declarative (like HTMX attributes and CSS transitions).

Симуляция подложки (Underbase Simulation)

- **Контекст:** В DTF-печати при печати на темных материалах сначала наносится **белая подложка**, затем цветной слой. Пользователям, особенно дизайнерам, важно понимать, как их дизайн будет выглядеть с этой подложкой – т.к. цвета могут меняться, белый цвет в макете может быть либо прозрачным (если нет подложки) либо покрыт белым и т.д. “Symulation Underbase” – функция, которая показывает пользователю области печати белым слоем.
- **Реализация идеи:** В Preflight (и, возможно, на других этапах) пользователь может переключить режим “Показать подложку”. При этом вместо цветного изображения отображается **маска подложки** – т.е. где будет напечатан белый. Обычно это все пиксели дизайна, которые не прозрачны, за исключением, возможно, тех, которые сами белые? (В

DTF, если дизайн содержит белый цвет, его печатают? Обычно да, печатают белым тоже, но зависит от pipeline.) Как минимум, маска = альфа-канал дизайна.

- **Способы показать:**
- *В виде отдельного цвета:* Например, заливаем все непрозрачные области сплошным ярко-фиолетовым или черно-белым узором – чисто для визуализации. Но лучше *показать как белый слой*: то есть поверх темного фона отобразить сам дизайн, но весь цвет заменить белым, сохранив непрозрачность. Это пользователь воспримет как "вот где ляжет белая краска".
- *Накладывать поверх оригинала:* Еще вариант – наложить полупрозрачный слой цвета (например, синего) на те области, которые будут с подложкой, чтобы видеть и цвет, и отметку. Но это может запутать. Проще toggle: оригинал vs маска.
- *Lens compare:* как упоминалось, очень уместно: половину изображения (или через линзу) показывать с режимом подложки, другую – обычную. Тогда сразу видно разницу.
- **Технически:**
- Если у нас загруженное изображение пользователя (PNG/SVG), генерация маски может быть либо на клиенте, либо на сервере. *Простой путь:* canvas 2D in browser:
    1. Draw user image onto canvas.
    2. Access pixel data, for each pixel with alpha > threshold > 0 (meaning something will print), set that pixel to white (255,255,255) with same alpha (to preserve anti-alias edges).
    3. Everything else (transparent background) remain transparent (so canvas background default maybe grey to show white? Actually, to show white on white background, better to have canvas background as maybe checker or black).
    4. Display that canvas or convert to DataURL for an img. This processing for typical images up to, say, 2000x2000 could be heavy (4 million pixels in JS), might cause a slight freeze. Offload: spawn a Web Worker with the pixel loop. Or utilize CSS if possible:
    5. If design is simple or vector (SVG), could apply `filter: brightness(0) saturate(0) invert()` combos to turn it white? For color images not straightforward, brightness(0) will make it black likely. Actually, there's `invert()` but that inverts color, not uniform to white.
    6. Another trick: if image has transparency, overlay it on white background but saturate to 0 (makes grayscale) and set brightness high, might get near white? But colored parts would become light grey, not guaranteed pure white unless original is pure black? Not robust.
    7. So likely direct manipulation is needed for correctness.
    8. Alternatively, have server generate mask as part of preflight (maybe the system that prints already knows where white underbase will be, could provide a preview image or vector).
- Considering Codex automation: easier to just incorporate a small JS to do canvas conversion on the fly when user toggles. Or if server returned an underbase mask image URL in preflight data, just display it.
- Perhaps simpler: display original design normally, and for mask view, show the design image with a CSS filter that makes everything monochrome white. But as said, not trivial with filters to guarantee pure white vs transparent.
    - If we assume design likely in PNG with transparency and not crazy colors: a quick hack: `filter: grayscale(100%) brightness(1000%) contrast(1000%)` might turn all non-transparent parts basically white. Need to test.
    - grayscale will remove hue, brightness 1000% will blow out to white, contrast 1000% also saturates difference. Possibly effective for typical images (dark ones become white too unfortunately? Actually brightness 1000% on any non-black pixel yields white).
    - If design has black parts, grayscale-> black, then brightness up huge might still produce white because brightness applies an add? Might result in white or light grey. Contrast 1000% will push mid tones to extremes (white or black).

    - Might just do invert(true) but invert black->white and white->black, which is wrong if design had white intended not to print? It's risky.
- Ok, likely best: do it precisely with canvas or rely on back-end.
- **UI Implementation:**
- Add a toggle switch or button "👁 Show Underbase" in Preflight.
- When activated, either:
    - Replace the preview image with mask image (client or server computed).
    - Or show lens where moving lens reveals mask while outside lens is normal (this we partly covered in lens tool).
    - Possibly allow side-by-side: if space, could show small thumbnail of mask next to color.
- If using lens approach, then technical part is just prepping mask as second image and hooking into lens component (the lens can have one image as mask, one as normal).
- We should caution: if image is large and doing this on client, do once and cache result (so subsequent toggles don't recalc). E.g., do calculation on toggle ON only first time, store in memory.
- On mobile older devices, skip this if performance too bad: we could detect if image resolution > some threshold and device memory low, then simply not provide (or warn "Underbase preview not available, trust our process" etc).
- **Performance:** If doing on client, do heavy computation in a separate thread (Web Worker). Alternatively, do a small scaled-down version for preview to reduce pixel count. (If actual print is huge 300 DPI image, don't attempt that full).
- Could pre-limit upload dimensions (like require user upload not huge or down-scale it for preview).
- Memory: converting to canvas uses memory roughly width*height*4 bytes, so e.g. 1000x1000 = 4e6 ~ 4 MB, fine. 4000x4000 = 64 MB, borderline. So might restrict preview to 2000px max dimension.
- **Precision:** The mask logic: all non-transparent -> white. If the print process would not put white under pure white areas? Actually, in DTF, you usually DO print white under everything that isn't blank, because even white color in design needs white ink on dark shirt; but if printing on white shirt, maybe no underbase needed at all. Possibly outside scope, but:
- If user selected garment color in order, the underbase simulation should consider it: e.g., if garment is white, then no underbase needed (maybe only under parts that aren't white in design? Actually if garment is white, white in design can be left as garment, some systems do that).
- It complicates logic. But at least handle dark scenario.
- Could mention: If printing on white material, the simulation may show no underbase or minimal.
- **Integration:** It's likely combined with "Compare/Lens" – mention that above.
- Possibly we implement underbase simulation as part of lens, but also allow full toggle to just switch image to mask.
- Up to design team, but codex should code both if needed.

Вставка из буфера (Paste-to-Order)

- **Назначение:** Ускорить процесс добавления изображения в заказ. Пользователь может скопировать изображение (например, из Photoshop или браузера) и **вставить (Ctrl+V)** прямо на страницу заказа, вместо того чтобы нажимать “Выбрать файл” и искать. Это улучшает UX для опытных пользователей и демонстрирует высокотехнологичность сервиса.
- **Реализация:**

- Используем Events API: слушаем событие `paste` на документе (или на конкретной зоне, например, на блоке загрузки файла). Когда происходит paste, проверяем `event.clipboardData`. Если содержит элементы типа image (PNG/JPEG) – извлекаем Blob.

```
document.addEventListener('paste', e => {
  const items = e.clipboardData.files || e.clipboardData.items;
  for (let item of items) {
    if (item.type && item.type.startsWith('image/')) {
       // we got an image file
    }
  }
});
```

  In Chrome, `clipboardData.items` could have `type = "image/png"` etc if an image was copied from an app. Or .files in some cases yields a FileList.
- If found, prevent default (so it doesn't paste as image element or do nothing).
- Then proceed to handle it like an upload: Possibly we have a hidden `<input type=file>` in the form, we can populate it with the blob? Unfortunately, can't just assign a File object to input (security restrictions). Instead, might directly send it via fetch or use HTMX to upload:
  - Could create a FormData, append file blob (need a filename, can default "pasted.png").
  - Send via HTMX: e.g. `hx-post` on a dummy `<div hx-post="/order/upload" hx-trigger="paste" hx-include="#order-form">` might not straightforward.
  - Simpler: use Fetch API in paste handler to POST to upload endpoint (which returns say JSON with file id or preview). But let's keep to HTMX style: we can programmatically call `htmx.ajax()` too.
  - Or we cheat: create an `FileReader` to preview image in UI instantly (display it in the preview area), and also store it in a global variable or attach to form via hidden data URL. But for actual submission to server, better to do it now:
  - Possibly better approach: use standard `<input type=file>` but with `webkitdirectory`? No, for paste no direct way to tie.
  - So likely manual AJAX: after paste, show preview image on page (like if input had been used), and either:
    - Immediately upload it to server (AJAX) so we get an ID, and include that in form (like hidden input with uploaded image ID). Many modern apps do that (upload as soon as you choose file).
    - Or store file blob in JS memory and intercept form submit to attach it.
  - The immediate upload approach is more responsive: user sees upload progress maybe, and we can show once done. If user cancels order, oh well image is stored temporarily.
  - For scope, we can let Codex do simpler: simulate file input selection: Actually, one hack: you can create a new File from blob and use DataTransfer to programmatically set in an <input> via events, but it's quite hacky.
  - Let's plan:
  - On paste, insert an `<img src=URL.createObjectURL(blob)>` in the preview area so user sees it.
  - Also, store `window.pastedFile = blob`.
  - On form submit (hx-post), if `window.pastedFile`, attach it via FormData manually in an event hook (like intercept default form submission, do fetch with FormData).
  - This might conflict with using HTMX for form normally (if hx-post on form, HTMX will do it, but we can override).

    - Alternatively, skip HTMX for main form submission if using custom logic for file. Or instruct Codex to incorporate this logic carefully.
- **Fallback:** Not all users know paste or not all mediums allow it (mobile can't paste images easily). So we keep the normal file input as primary. Paste just supplements. Also if paste fails (no image in clipboard), either do nothing or show a tooltip "Ничего не найдено в буфере".
- We should highlight that feature in UI: maybe a dashed border box "Вставьте изображение сюда или нажмите для выбора файла". So user knows it's possible.
- **Security:** Accepting pasted image is similar to uploaded file. But ensure to restrict type (only image types).
- If someone pastes giant image, we might want to compress client-side before sending. Could optionally downscale if enormous, but probably not necessary since presumably they'd paste what they'd upload anyway.
- Also ensure not to double-upload the same thing inadvertently (if user spams paste).
- **Performance:** Paste is one-time event, not a big overhead. But previewing large images via URL.createObjectURL is fine. If we want, we can draw on canvas to resize if needed (like 4k -> 1k preview).
- Make sure memory freed: revokeObjectURL when not needed or after form submission.
- **Codex Implementation details:**
- Should use JS event for paste (no direct HTML support).
- Possibly include a small `<script>` in page for that feature.
- Use modern JS and minimal dependencies (no jQuery).
- Also consider if multiple images could be pasted at once (if user copies multiple files?). We can decide to only handle one (first).
- Provide user feedback: after paste, maybe show "Image attached" in UI. Possibly simulate the file input listing the file name.
- **User feedback:** Could integrate with Preflight seamlessly: e.g., after paste and upload, directly jump to Preflight step since image ready, skipping manual "Next". But that might surprise user. Better to just fill the form field and let user click "Next" as normal.

Инструмент сравнения (Compare/Lens)

(См. раздел «Компоненты UI – Линза сравнения» для дизайн-аспектов.)

- **Подключение на конкретных кейсах:** Инструмент реализуется как независимый модуль JS/CSS, который можно включать на разных страницах:
- В **галерее работ**: при открытии модалки кейса, можем показать “оригинал vs фото” – тут lens показывается сразу внутри модалки, разделяя оригинальный макет (например, изображение, которое печатали) и фотографию готового изделия. Это хорошо демонстрирует качество печати.
- В **Preflight:** как обсуждалось, lens может сравнивать “без подложки vs с подложкой” или “макет vs как на материале”. Например, если у нас есть графический эффект на превью (скажем, мы имитируем текстуру ткани), тогда lens может сравнить плоский макет и с учетом текстуры. Но пока у нас точно есть сценарий подложки.
- Возможно, lens можно применить и на странице «О компании» или «Как это работает», если там демонстрируем отличие, но это уже вторично.
- **Внедрение HTML:** Нужно два изображения или два блока. Верстка для ползунка варианта:

```
<div class="compare-container">
  <div class="compare-img compare-img-a"><img src="original.jpg"
alt="Оригинал"></div>
  <div class="compare-img compare-img-b"><img src="printed.jpg"
```

```
alt="Результат"></div>
  <input type="range" class="compare-slider" value="50" aria-
label="Сравнение">
</div>
```

This slider version is simpler for implementation. The `compare-img-b` can be absolutely positioned on top and width controlled via slider: `compare-img-b { width: <slider>% }`. That yields a vertical wipe effect. If want a circle lens, more complex:

- Perhaps simpler plan: Use slider on desktop, and maybe circle lens on hover (some fancy, but we can skip circle if time).
- But the user explicitly wrote "линзы", likely they want the fancy circular lens, as it's cooler. We'll attempt it:
    - Approach: have a circular `div class="lens"` with overflow hidden, which contains second image. This lens is absolutely positioned and moves with JS (on mousemove relative to container).
    - The container shows first image normally.
    - On event (mouseover or toggle button), we append lens element at default position (center).
    - On mouse move, update lens position to cursor (with some offset so cursor at center maybe).
    - On touch (mobile), maybe not use lens because no hover, or use slider fallback. Possibly easier: on mobile always show slider, on desktop allow lens by mouse.
    - Provide a toggle between modes if needed (or just decide one mode).
    - The slider variant can be implemented purely CSS (input range) plus a bit of style.
    - The circular lens requires JS but maybe better visual and more WOW. Up to design preferences.
- Codex can implement one robustly. Possibly slider is more accessible and simpler. But "линзы" implies the circular metaphor. Could do both: default slider (accessible, always visible control), and also allow moving circle if user hovers (like a fancy addition).
    - That might confuse though.
- Might stick to slider for practicality in code, except if "линза" explicitly requested meaning circular.
- **JS for circular lens:**
- Listen to container `mousemove` and `touchmove`.
- Position lens div (set style.left/top = event.x - radius).
- Possibly some constraints so lens doesn't go outside container fully.
- If performance issues with continuous move, can throttle to requestAnimationFrame (should be fine).
- Add event `mouseleave` to hide lens or reset position? Or keep last position.
- Optionally allow click to freeze lens or to toggle off.
- **CSS for lens:** lens div has `width:100px; height:100px; border-radius:50%; border: 2px solid #fff; overflow:hidden; pointer-events:none (so it doesn't block cursor events to container itself)`.
- Put second image inside lens with position relative such that when lens moves, second image moves oppositely to align correctly:
    - e.g. if lens at (x,y), second image inside lens should be at (-x, -y) to show the part of second image corresponding to that area. But container images must align background wise.
    - It's easier if both images are same size and superimposed in same coordinate system.
    - So we might position container as relative, second image as absolute full size but clipped by lens container. Actually:

    - Put both images absolutely on top of each other.
    - For lens approach, hide second image globally except lens region: ~ Could use CSS clip-path: e.g. `clip-path: circle(10% at X Y);` on second image. Actually, that's a clever way: dynamically update clip-path center to cursor, radius equal lens radius. Then the second image automatically shows only that circle. That saves needing a separate lens element and moving image inside it, you directly mask it. ~ But CSS clip-path dynamic update with JS may cause reflow but should be okay. Or even use CSS custom properties and update those (like `--cx: 100px; --cy: 150px; clip-path: circle(50px at var(--cx) var(--cy))`). ~ This is an approach: one composite image, second shows through circular mask at cursor. It will appear exactly as lens with border if we also add border via another element or use outline with clip? Possibly easier to physically have an element.
    - Maybe simpler: have lens element and inside it place second image tag referencing same source as the second image we want to show. Then update that image's `margin-left = - (lensX)`, `margin-top = - (lensY)` such that what's visible inside lens is the correct portion.
    - The drawback: if second image is large, moving it means each move maybe triggers re-render of large image. But likely it's fine if using GPU for transform.
    - Alternatively, use `background-image` on lens div referencing second image and use background-position to simulate movement (less layout thrash).
        - Actually, that might be neat: lens div with `background: url(secondImage) no-repeat; background-size: contain;` (assuming second and first images same size).
        - Then on move: set lens.style.backgroundPosition = `-xpx -ypx`.
        - That might be more efficient than moving an <img>.
    - Ensure quality (if retina, background might pixelate? But if image loaded with high res it should be fine).
- Considering time, I'd choose slider method in code because it's simpler to get right quickly and accessible (range input).
- But slider is less "wow" perhaps. Possibly do slider with a toggle to circle for advanced users?
- Could mention both. The question implies advanced technique, maybe they expect the fancy lens.
- **Performance:** Slider is trivial to animate (just one CSS width update on drag).
- Circle lens needs continuous tracking, but still should be fine for modest image sizes.
- Use transforms for background move to leverage GPU if possible.
- On mobile, avoid heavy lens because dragging finger might cause more jank.
- Provide an option to disable lens if it lags (like revert to static compare).
- **Integration with metrics:** Could track usage of compare feature (like how many toggles).
- **Fallback:** If JS off, obviously lens won't work; we can just show both images one under other with labels "Before / After".
- Make sure alt text explains difference so it's accessible.

<!-- We have covered technical aspects for each item. Now the remaining sections: 7, 8, 9, 10 with restrictions, metrics, content, format. -->

7. Ограничения для Codex-агента

При генерации кода и логики необходимо соблюдать ряд ограничений и правил, чтобы итоговый продукт соответствовал требованиям дизайна, был безопасным и устойчивым:

- **Никакого самотворчества вне ТЗ:** Агент **не должен придумывать новые функции или эффекты**, не описанные в этом документе. Каждая анимация, каждый интерактивный элемент должен соответствовать оговоренному сценарию. Запрещено добавлять "веселые" пасхалки, скрытые сообщения, несогласованные визуальные элементы. Творческий подход приветствуется только в рамках заложенных концепций (например, можно варьировать продолжительность анимаций в разумных пределах, но **нельзя** добавлять новую 3D-анимацию, о которой нет речи). Вся функциональность должна быть предсказуемой для команды.
- **Соблюдение дизайна vs. fallback:** Если какая-то фича технически не поддерживается в среде пользователя, агент должен автоматически откатиться на запасной вариант, **не ломая UX**. *Примеры:*
- Браузер не поддерживает `ViewTransitionsAPI` – тогда переходы страниц происходят стандартно (можно при желании просто мгновенно без плавности, или с базовым CSS-решением).
- `prefers-reduced-motion` включен – все анимации уровней L2+ отключаются.
- `backdrop-filter` недоступен – стеклянный эффект заменяется на просто прозрачный темный фон.
- Не удалось отрендерить "Printhead Scan" (canvas WebGL error или что-то) – вместо него просто появляется изображение сразу (и консоль может залогировать предупреждение, но без вываливания ошибок).
- Пользователь старого IE (теоретически) – сайт должен хотя бы отобразить контент (пускай без стилей и скриптов возможно, но чистый HTML со всей информацией).
- Агент **должен предусматривать** все такие случаи. Необходимо обрабатывать ошибки (try/catch в критичных местах JS), иметь условия на фичи.
- **Безопасность и данные:** Нельзя использовать потенциально уязвимые конструкции. Например, никакого `eval` в JS, никакого прямого использования непроверенных пользовательских данных в DOM (XSS уязвимости). Все входные данные (особенно файлы, текст) должны либо проверяться на сервере, либо экранироваться. Codex-агенту запрещено генерировать код, который отправляет данные на сторонние сервисы без разрешения. Аналитика – только то, что оговорено (см. Метрики).
- **Минимизировать зависимости:** Использовать **чистый HTML/CSS/JS** или легковесные библиотеки. Запрещено подключать тяжелые фреймворки (React, Angular, etc.) – у нас MPA+HTMX подход. Также не нужно тянуть большой UI-кит, если можно реализовать в нескольких строках. Допустимо подключить небольшие утилиты если крайне необходимо (например, polyfill для ViewTransition, но лучше graceful degrade).
- **Производительность превыше лишних украшательств:** Если в процессе реализации окажется, что какой-то анимационный эффект **значительно нагружает** CPU/GPU или влияет на Core Web Vitals, агент должен **уменьшить/отключить этот эффект**, даже если в дизайне он желателен. Важно соблюдать метрики (см. далее). Например, если анимация параллакса начинает фризить на мобильных – нужно автоматически отключать ее при `window.innerWidth < X` или `devicePixelRatio < Y`.
- **Доступность обязательна:** Ни один шаг не должен нарушать доступность. Агент **не имеет права** убирать фокус-обводки, отключать tab-навигацию или заменять нативные элементы неэквивалентными (например, делать `<div>` вместо `<button>` без роли). Все ARIA-атрибуты должны добавляться как нужно. Проверять контраст при динамическом обновлении стилей.

- **Строгость к тестам:** Генерируемый код должен быть протестирован (или легко тестируем). Агент не придумывает “на глазок” – использовать инструменты верификации: валидатор HTML, wave (для a11y), Lighthouse для перформанса. Если метрики не проходят – править.
- **Контент и текст:** Агенту запрещено менять тексты, слоганы, названия без указания. Весь контент (особенно маркетинговый на главной) должен браться из утвержденных источников. Если таких данных нет – оставить заглушки/комментарии для копирайтера, но не генерировать от себя.
- **Конфиденциальность:** При работе с user-upload (файлы) – не сохранять их навсегда без надобности, не посылать чужим. При интеграции сервисов (не оговорено явных, но вдруг) – соблюдать минимально необходимые разрешения.
- **Запрет на нарушение логики заказа:** Очень важно, чтобы любые вау-эффекты **не влияли на процесс заказа**. Т.е. ни Printhead Scan, ни HTMX обновления не должны мешать отправке заказа, повторно сабмитить или сбрасывать данные. Агент должен тщательно продумать состояние формы: например, если частично обновляем форму через HTMX, не сбрасывать поля, сохранять введенное. Лучше лишний раз проверить, чем потерять данные пользователя.
- **Документирование в коде:** Агенту предписано генерировать код с комментариями (как указано в разделе Формат), которые поясняют сложные места, *но* эти комментарии не должны попадать конечному пользователю или тормозить выполнение. То есть JS/HTML-комментарии – ок (они не влияют), но не console.log с отладкой (если продакшен).
- **Не нарушать внешние интеграции:** Если сайт подключается к платформе (CMS или backend) – не перекрывать существующие стили/скрипты без понимания. Например, если Twocomms.shop использует WooCommerce или Django, нужно встроиться аккуратно, не ломая их JS.
- **Правила SEO:** Не вводить ничего, что повредит SEO (например, все важные тексты должны быть в HTML, а не рисоваться на canvas). Никаких скрытых текстов (кроме aria-only).
- **Юридические моменты:** Агент не создает контент, который нарушает чьи-то права. Например, если вставляем сторонние шрифты/иконки – проверяем лицензию (Google Fonts are fine). Изображения для дизайна – либо предоставлены, либо бесплатные.

Сводя воедино: Codex-агент генерирует только то, что нужно, и ничего лишнего; любые отклонения – в сторону упрощения, а не добавления нового.

<!-- Codex: Do NOT generate extraneous code or features. Follow the spec exactly. Include all fallback logic as described. Always prefer clarity and stability over fancy but risky shortcuts. -->

8. Метрики и проверка успеха

Чтобы убедиться, что редизайн приносит пользу и не вредит юзабилити, мы определяем набор метрик для отслеживания. Codex-агент должен в коде предусмотреть возможность сбора этих метрик (через data-атрибуты, events, интеграцию аналитики), а команда – периодически анализировать их. Также определяем, какие A/B тесты допустимо проводить на новой версии.

- **Производительность (Web Vitals):** Ключевые метрики загрузки и отзывчивости:
- *Largest Contentful Paint (LCP):* должно оставаться **< 2.5с** на десктопе и мобильных для главного контента (цель – хороший рейтинг [5]). Внедрение View Transition не должно сильно увеличивать LCP (ориентируемся, что +70мс на мобиле приемлемо [5], если больше – пересмотрим использование).

- *First Input Delay (FID) / Interaction to Next Paint (INP):* интерактивность не должна страдать от длинных задач. Следим, чтобы тяжелые JS (например, обработка изображений) были разбиты или выполнены в воркерах. Цель – FID < 100мс.
- *Cumulative Layout Shift (CLS):* дизайн должен быть стабилен, не прыгать. Предзадать размеры изображений, шрифтов. CLS целевой < 0.1.
- *Frames Per Second (FPS):* при анимациях, особенно L3–L4, стараться держать 60fps на средних устройствах. Минимум 30fps как нижняя граница, иначе анимацию упростить. Для диагностики можно закладывать измерения (например, `requestAnimationFrame` count vs setInterval).
- *Memory usage:* особенно при canvas и больших картинках – наблюдать, не растет ли непрерывно. Проводить профилирование, избегать утечек (например, `URL.revokeObjectURL` после использования, удалять DOM элементов по завершении анимаций).
- *Server response time:* HTMX запросы должны быстро возвращать фрагменты (сервер оптимизация, вне зоны фронтенда, но агент может уменьшить payload – не запрашивать лишнее).
- **Конверсия и поведение пользователей:**
- *Conversion Rate:* Главный показатель – доля посетителей, оформивших заказ. Цель редизайна – **не снизить конверсию**, а лучше повысить (например, +5% через улучшенный UX). Нужно отслеживать CR старой vs новой версии. Codex-агент, конечно, не контролирует это напрямую, но должен предоставить чистый путь к целевому действию (CTA видны, ничего не ломается).
- *Bounce Rate:* В идеале, “вау”-дизайн снизит bounce (люди остаются дольше). Следим, чтобы bounce не вырос – если вдруг пользователи уходят, может что-то отвлекает или долго грузится.
- *Время на сайте, страницы за сессию:* Возможно увеличится, если интересно смотреть галерею (это нормально). Но время на ключевых шагах (например, оформление заказа) не должно значительно увеличиться – иначе юзер буксует. Можно мерить время от входа на Order до нажатия “Submit”. Если слишком долго, может интерфейс усложнил – надо будет смотреть.
- *User engagement с WOW-элементами:* Метрики типа “% пользователей, которые взаимодействовали с Printhead Scan или Compare Lens”. Если очень мало, возможно эти фичи незаметны или неудобны. Такие события можно слать в аналитику (on animation start or toggle click send event "UsedPrintheadScan").
- *Ошибка/брошенные формы:* Если пользователь начал заказ но не закончил – важно отслеживать. Может, какая-то часть UI мешает (например, модалка подтверждения нелогичная). Так что track drop-off rate per step.
- **Доступность (Accessibility metrics):** Можно подключить инструменты, отслеживать наличие проблем:
- ARIA-coverage: % элементов с нужными aria-метками (manually audit).
- Контраст: автоматически можно проверять через CI (Lighthouse a11y score). Цель – 100% a11y score Lighthouse.
- Пользовательские отзывы: Внедрить форму быстрого фидбека: “Оцените удобство сайта” – если возможно, для реальных людей, собирать qualitative feedback.
- **A/B тесты:** Допустимые эксперименты:
- **Интенсивность анимаций:** Можно делать A/B, где группе A показываем полный WOW (все L3–L4), группе B – урезанный (все L2). Сравнить конверсию и engagement. Если разницы мало, можно снизить анимации по умолчанию (экономия перформанса). Или наоборот, если вау-группа конвертит лучше, оставить или усилить.

- **CTA дизайн:** Например, тестировать разные цвета или тексты кнопки “Заказать”. Но **важно:** такие тесты не должны менять общий стиль радикально. Нельзя тестировать “плохой vs хороший дизайн”, только тонкие изменения, чтобы не портить бренд.
- **Контент раскладки:** Порядок секций на главной – возможно тест (вдруг лучше сначала не преимущества, а сразу галерея?). Но это скорее стратегический, агент отмечает, что структуру можно перетасовывать, но не делает сам без указания.
- **Performance vs Effects:** Например, тест: включен ViewTransition vs выключен, замерять engagement. Или fancy Preflight vs simple file upload (вдруг пользователям сложно с fancy?). Через A/B можно понять влияние.
- **Не разрешены A/B:** Все, что противоречит ядру бренда или может запутать: нельзя тестировать совершенно разный визуальный стиль, нельзя одним пользователям давать упрощенную форму, другим сложную (разве что если не уверены, но обычно форма должна быть оптимальной сразу).
- Тесты должны быть этичными: не скрывать цену у одной группы и показывать другой, и т.п.
- **Мониторинг и алерты:** Агент должен закладывать, что после релиза нужно мониторить:
- Ошибки JS (window.onerror – посылать в лог/сервис).
- Падение сервера (DevOps side).
- Core Web Vitals (есть API send to Google Analytics, Codex может встроить snippet).
- Если метрики выходят за рамки (например, LCP > 4s у многих) – алерт команде для оптимизации.
- **Версии и откаты:** Продумывается план отката на старый дизайн, если метрики резко ухудшатся. Поэтому агент генерирует новый код **модульно**, чтобы можно было параллельно держать старый, и переключение (feature flag) несложно.
- **SEO метрики:** Проверить, что редизайн не уронил трафик (это уже следствие качества кодировки: заголовки H1-H2 присутствуют, тексты индексятся, скорость не просела).
- **User Testing:** Хотя это не числовая метрика, заложим, что перед полным запуском провести UX-тесты 5-10 людей. Агент может не участвует, но вся структура документа помогает в таких тестах (каждый компонент можно проверять “заметили ли пользователи Printhead Scan, понятна ли форма” и т.д.).

Codex-агент в коде может помечать места для метрик (например, `data-analytics="hero-cta"` на кнопке заказать, чтобы аналитики настроили отслеживание кликов). Целевые показатели должны быть достигнуты, либо мы быстро узнаем и исправим с минимальными потерями.

<!-- Codex: Ensure to include data attributes or hooks for analytics where needed (e.g., data-event labels on major buttons), and keep code performance-friendly to hit Web Vitals thresholds. -->

9. Контент и медиа материалы

Визуальное и текстовое наполнение играет огромную роль в успехе дизайна. Вот требования к контенту, который будет использоваться, и указания по его подготовке:

- **Фотографии продуктов и кейсов:** Обязательно использовать **реальные фото** напечатанных работ в высоком качестве. Для галереи и главной нужно провести фотосессию: примеры футболок, худи, патчей с отпечатанными дизайнами TwoComms. Стиль фото:
- На нейтральном фоне (лабораторный стол, чистый серый/черный фон) – чтобы фокус на принте.
- С правильным освещением: рекомендуется мягкий заполняющий свет + акцентный свет сбоку для объема, без резких теней. Печать должна быть четко видна, цвета насыщены.

- Макро-снимки: 2-3 фото прям близко к ткани, чтобы показать текстуру печати, тонкость линий – для раздела “Quality/Lab Proof”.
- Контекстные фото: люди в мерче на улице или в интерьере – можно 1-2 для разнообразия, но основной упор на продукт. Если используются люди, согласовать, чтобы образы соответствовали бренду (легкий стритвир, techwear).
- Все изображения оптимизировать для веб (не грузить 10Мб фото; подготовить версии 1x и 2x for retina, WebP format).
- **Изображения процесса (лаборатории):** Чтобы усилить идею Lab Proof, нужны фотографии/видео производственного процесса:
- Принтер DTF в работе: крупный план печатающей головки, кадр как она накладывает слои (можно даже попытаться сделать короткое видео для hero).
- Лабораторное окружение: стол с химическими банками (краски), человек в перчатках контролирует процесс – создает ощущение, что все профессионально.
- Thermopress, пленки с дизайнами, проверка качества под лупой – любые такие кадры уместны в разделе “Как мы это делаем” или фоново.
- **Видео:** Если есть возможность, идеально сделать **15-20 секунд видео** для фонового использования в hero: slow-motion, как головка проходит по пленке и выдает яркий рисунок. Сделать его без звука, затемнить слегка если на фон, и оптимизировать (webm/mp4 in moderate resolution). Видео должно быть **ломаемым**: т.е. если устройство слабое или user scrolled, остановить/пауза.
- **Графические элементы UI:** Иконки, иллюстрации:
- Иконки для: доставка, качество, гарантия, иконки этапов статуса, иконка “вставить” для Paste-to-Order (например, значок буфера обмена), иконка сравнения (две половинки круга).
- Стиль иконок: линейные или монохромные, с тем же акцентным цветом если нужно. Должны быть **единого набора** (лучше использовать библиотеку типа FontAwesome Pro или custom SVG, но привести к одному стилю).
- Иллюстрации: возможно, фоновые графики: абстрактные tech-узоры (сетки, схемы) еле заметно на фоне, чтобы добавить глубины (можно SVG pattern с прозрачностью). Не перегружать, но 1-2 таких может украсить фон секции.
- Логотипы партнеров/клиентов: Если есть значимые клиенты, можно вывести их лого. Но они должны быть в белом/color одном стиле, выровнены. Не бросать просто так – либо в секции “Нам доверяют” либо в футере.
- **Текстовый контент:**
- **Заголовки и слоганы:** Короткие, емкие. Пример: “Преврати идею в прочный принт” на главной. Codex-агент не генерирует финальный маркетинг текст – это работа копирайтера. Но в коде и шаблонах, оставить места для них. Например, `<h1>{{ main_slogan }}</h1>` с примечанием “Слоган от маркетинга”.
- **Описания:** На странице заказа – подсказки к полям (например, “Формат: PNG, до 50 МБ”), на Preflight – объяснение “Проверьте макет, так он будет выглядеть”. Эти тексты должны быть дружелюбны, без канцелярита. Тон бренда: на “ты” или “вы” – решить и придерживаться. Судя по стилю TwoComms (из описаний товаров мы видели), вероятно, на “ты” с молодежным дерзким оттенком. Но на сайте лучше чуть более нейтрально, но всё же “ты” допускается (“Загрузи свою работу и увидь магию печати!”).
- **Микрокопирайтинг:** Надписи на кнопках, всплывающих сообщениях – четкие и однозначные. “Заказать сейчас”, “Загрузить макет”, “Добавить в корзину”, “Отменить”. Если Printhead Scan имеет кнопку – назвать ее не техническим термином, а понятно: “🖨 Симулировать печать”.
- **Ошибки и уведомления:** Пользовательские ошибки (например, “Слишком большой файл”, “Неподдерживаемый формат”) – фразы, объясняющие что не так и как исправить.

Т.е. “Файл больше 50 МБ. Пожалуйста, загрузите файл поменьше или свяжитесь с нами для крупного заказа.”

- **Мультиязычность:** Если сайт будет двуязычным (англ/укр/рус) – тексты должны браться из локализации. Codex-агенту нужно внедрить механизмы i18n, но конкретные тексты пусть будут на языке original (в данном случае, видимо русский или укр?). Возможно, dtf.twocomms.shop ориентирован на укр рынок, однако задание на русском – предполагаю локаль RU. Может быть нужно RU и UA версии. Надо уточнить у заказчика. Но надо предусмотреть, если что.
- **Видео и анимации на сайте:**
- Как упоминалось, видео с печатью – если есть, используем. Также можно использовать **анимированные SVG** для мелочей: напр., значок загрузки (спиннер) кастомный, анимирован CSS или SVG (не гиф, гиф тормозной).
- Lottie файлы: если дизайнеры сделают лотти-анимацию (например, логотип мерцает) – агент может вставить lottie-web runtime. Но это доп.библиотека ~50KB, только если очень надо.
- **View transitions (Pseudo-content):** One hack: we can set an image as a pseudo-element to animate it between pages. But maybe not needed.
- **Где использовать контент:**
- Главная: Большой заголовок + фон (возможно видео/изображение). Далее блок “Преимущества” – здесь текст + иконки. Блок “Как работаем” – пошагово (текст + иллюстрация). Блок “Примеры работ” – фотогалерея (с возможностью открыть).
- Order: минимум отвлекающих картинок – можно маленькую иллюстрацию рядом с “Загрузите файл” (иконка файл). Но в целом, форма = текст+контролы.
- Status: можно добавить небольшой декоративный элемент, напр. анимированная иконка принтера наверху, которая меняется с состоянием (печатает -> готово). Только если не отвлечет.
- Gallery: сетка фото (сам контент = фото).
- Preflight: превью – это сам контент (изображение). Можно где-то на фоне еле заметно контур принтера.
- Футер: туда можно поместить еще немного контента: например, повторить контакты, соцсети, небольшое фото принтера в полупрозрачном виде, чтоб стилизовать футер. Но не обязательно, можно просто цветной блок.
- **Съемка и создание нового:** Если какого-то контента не хватает (скажем, нет приличных фото), лучше его создать, чем ставить сток бездумно. Стоковые фото будут выбиваться. TwoComms явно бренд со своей атмосферой, поэтому контент должен быть аутентичным. Команде стоит запланировать **создание недостающего контента параллельно разработке**:
- Снять видео печати (да, нужно оборудование, но хотя бы на телефон стабильно).
- Сделать макеты (может, 3D mockup) для мест, где фото нельзя.
- Подготовить графику для Printhead Scan – может, отдельный слой изображений (например, картинка полосы).
- **Оптимизация контента:** Все медиа должны быть оптимизированы:
- Использовать современные форматы: WebP, AVIF для изображения, fallback JPEG for older.
- Responsive <img>: разные размерности (srcset) чтобы мобильным отправлять меньший файл.
- Lazy-load: Галерею грузить постепенно (можно `loading="lazy"`).
- Видео – сжатое, short, loop if needed, with `playsinline autoplay muted` attributes for background usage. Or controlled via JS to play once.
- Test content on retina vs non-retina – ensure no blurriness.
- **Контентные сценарии (кейсы):** Помимо медиа, возможно, нужны текстовые кейсы: краткие истории или цифры (например “500+ заказов успешно выполнено за 2023”). Такие

вещи можно вставить в главную страницу для солидности. Если упоминаем их, проверить достоверность.
- **User-generated content:** Если планируется раздел, где пользователи сами загружают (галерея клиентов), то нужно модерацию. Но пока не указано.
- **Документы и файлы:** Если есть страница “Примеры шаблонов” или "Документация" – контент (pdf, template .PSD) – выложить и дать ссылки. Но это далеко от UI/UX, просто не забыть, если надо.
- **Тон коммуникации:** Контент (визуальный и текст) должен отражать позицию TwoComms: судя по названиям товаров (“Довіряй своїй божевільній ідеї” etc.), бренд дерзкий, мотивирующий креативность, с отсылками к свободе, революционности. На сайте это можно подкрепить:
- Цветовая гамма смелая (черный + яркий акцент, видимо),
- Слоганы под стать: “Дай волю своей безумной идее – мы напечатаем ее на реальности” (грубо говоря).
- Но балансируем, чтобы не отпугнуть деловых клиентов – сохраняем профессионализм (через "лабораторность", техничность).
- **Медиа лицензирование:** Убедиться, что на все фото/видео TwoComms имеет права. Если используем стоки или чужие материалы (например, иконки) – лицензия должна позволять коммерческое использование. Лучше открытые библиотеки (Feather icons etc.).
- **Проверка перед запуском:** Прогнать весь контент через разные устройства: нет ли тяжелых элементов на мобильном (возможно, отключить видео на моб), читается ли мелкий текст на фото, корректно ли работают captions.
- **Контент-стиль гид:** Завести документ, описывающий правила контента, чтобы любой редактор или контент-менеджер знал, какие фото подходят, как писать тексты. (Это больше к внутренней документации компании, но упомянуть ценно.)

Codex-агент не отвечает за создание медиа, но его шаблоны и код должны быть готовы принимать эти материалы: предусмотрены нужные контейнеры, с правильными классами, возможность легко заменить заглушки финальными фото. Все demo-контент (латинская рыба, placeholder.jpg) потом заменить реальным перед продакшеном.

<!-- Codex: Use semantic HTML for content placeholders. E.g., <figure><img src="..." alt="описание"><figcaption>...</figcaption></figure> for images in gallery to allow captions. Prepare <video> tags if needed with appropriate attributes. Ensure text content is not hardcoded if it should come from CMS or translation files. -->

10. Формат и структура выдачи (для Codex)

Этот технический документ структурирован в формате, близком к **README.md/PROMPT.md**, чтобы Codex-агент мог напрямую использовать его как инструкцию. При генерации production-кода следует придерживаться аналогичной структурированности и богатого комментирования, чтобы код был легко читаем и поддерживаем разработчиками. Ниже – требования к форматированию итогового кода и ресурсов:

- **Markdown-документация:** Данный файл послужит основой для системных подсказок Codex. В нем мы использовали заголовки, списки, выделение **жирным** для терминов – подобно тому, как нужно документировать и сам код. Рекомендуется поместить этот файл как `README.md` в репозиторий проекта. Он будет полезен и как справка команде. Codex при генерации кода может ссылаться на разделы этого документа (например, указывать в комментариях “// См. раздел Motion tiers (L0-L4) выше”).
- **Комментарии в коде:** Требуется тщательно комментировать сложные места:

- В HTML шаблонах – использовать комментарии `<!-- ... -->` для объяснения блоков, которые могут быть неочевидны. Например: `<!-- Printhead Scan effect canvas: see main.js for animation -->`.
- В CSS – комментарии `/* ... */` около нетривиальных стилей (например, почему выбрано именно такое значение свойства, ссылки на источники, если это хак).
- В JS – `// ...` одиночные или `/** ... */` блоки для функций: описать, что делает функция, какие параметры, где используется. Особенно в модуле сравнения, paste-to-order, и т.д.
- **Стиль комментариев:** Должен быть понятным человеку, возможно двуязычным (рус/англ). Основной язык проекта может быть английский в коде, но если команда русскоязычная, можно дублировать кратко по-русски. (Это нужно уточнить с лидом. Часто код – на англ.) Лучше придерживаться английского в код-комментариях для международной практики, а в README можно и по-русски.
- Кодекс-стиль: избегать лишних извинений или несерьезного тона. Комментарии должны быть по делу, иногда с friendly заметками.
- **Чистота и логика структуры:** Разбить код на логические части:
- CSS – возможно использовать SCSS/LESS (в проекте, если доступно) организовать по компонентам: например, `_buttons.scss`, `_animations.scss`. Если Codex может генерировать сразу скомпилированный CSS, то хотя бы сгруппировать.
- JS – модульность: один файл для общих скриптов (`main.js`), и дополнительные для features (например, `printheadScan.js`, `compareLens.js`). При генерации, Codex-агент может сначала предоставить структуру, а затем заполнить функции – итеративно.
- HTML – шаблоны возможно разделены на части (например, с использованием шаблонизатора jinja2, blade, etc.), но Codex-агент может сгенерировать пример цельной страницы, помечая компоненты.
- Файловая структура:

```
/css/
  styles.css (собранный)
  components/
     buttons.css, form.css ...
/js/
  main.js
  printheadScan.js
  compareLens.js
index.html (Home)
order.html
status.html
gallery.html
preflight.html
```

  (Если серверный рендер, то .html заменятся на шаблонные .twig, .erb, etc., это уже адаптируется.)
- **Naming conventions:** классы CSS BEM или camelCase? Лучше BEM или подобный (например, `.btn--primary`, `.card__title`). Codex-агенту определить и держаться consistently.
- **Semantics:** как отмечалось, использовать правильные теги (`<header>, <nav>, <main>, <section>, <footer>`). Screen-reader only элементы для помощи.

- **Документирование специфичных решений:** Если пришлось принять решение (например, отключить feature X на mobile) – зафиксировать это в комментарии или README, чтобы все знали.
- **Примеры кода:** Если в README нужны примеры использования – например, HTML разметка компонента, CSS подключение – их оформить в markdown-блоках с указанием языка (`html`, css, ```js). Это поможет при обсуждении.
- **Стиль кода:** Автоформатирование (пробелы, отступы 2 или 4 – решить и везде соблюдать). Именование переменных JS – понятные (не `x` и `y` кроме коротких loop).
- **Проверка форматирования:** Перед выдачей кода, запустить линтеры (ESLint, Stylelint) – убедиться, что нет ошибок стиля или потенциальных багов. Codex-агент может сам быть настроен на “strict mode”.
- **PROMPT.md:** Возможно, после финализации ТЗ, часть этого документа может послужить prompt для Codex-агента 5.2 ExtraHigh. То есть, система будет давать агенту эти инструкции при генерации. В этом случае крайне важно, чтобы все перечисленное было предельно ясно: агент 5.2 – продвинутый, но следует точно формулировкам. Поэтому мы старались писать однозначно.

Наша итоговая цель – **генерация кода** на основе этого ТЗ. Если документ полон и понятен, Codex-агент сможет без двусмысленностей писать код, который: - соответствует описанной архитектуре, - включает все необходимые комментарии, - легко масштабируется и поддерживается, - и, конечно, реализует “вау”-дизайн без потери конверсии.

Таким образом, данный `README.md` / `PROMPT.md` является и техническим заданием, и частью системы Continuous Deployment (Infrastructure as Code, но в данном случае Design as Code). Любые изменения требований должны отражаться в этом документе, чтобы Codex всегда имел актуальную инструкцию.

<!-- Codex: Treat this document as the single source of truth. The code generation process should reference section numbers when needed (in comments) to justify decisions. Ensure the output code is well-structured and commented as per these guidelines. -->

---

1 2 3 4 The Moss REVAMP
https://www.themossstudio.ca/insights/wow-design-vs-conversion-finding-the-balance

5 6 7 The Impact of CSS View Transitions on Web Performance
https://www.corewebvitals.io/pagespeed/view-transition-web-performance

8 Less htmx is More
https://unplannedobsolescence.com/blog/less-htmx-is-more/